



**Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών
Υπολογιστών**

Εξάμηνο 6^ο: Βάσεις Δεδομένων

Εξαμηνιαία Εργασία:

**Σύστημα αποθήκευσης και διαχείρισης των πληροφοριών που
απαιτούνται για τη λειτουργία του Γενικού Νοσοκομείου
«Υγειόπολης»**

Αργύρης Πέτρος (el22805)

Καράπα Αθηνά (el23124)

Τασσόπουλος Πολυζώης (el23094)

Αθήνα
2026

Περιεχόμενα

1. Εισαγωγή.....	3
2. Περιβάλλον υλοποίησης.....	3
3. Παραδοχές και σχεδιαστικές επιλογές.....	4
4. Διάγραμμα ER.....	4
5. Σχεσιακό διάγραμμα.....	6
6. Υλοποίηση Βάσης.....	8
7. Περιορισμοί & Triggers.....	9
8. Ευρετήρια (Indexes).....	13
9. Φόρτωση δεδομένων.....	19
10. Ερωτήματα (Queries).....	19
11. Web εφαρμογή.....	45
Αναφορά χρήσης LLM (Large Language Models).....	45
Παράρτημα: DDL script.....	46

1. Εισαγωγή

Η παρούσα εργασία αφορά τον σχεδιασμό και την υλοποίηση μιας βάσης δεδομένων για το Γενικό Νοσοκομείο «Υγειόπολης». Στόχος της βάσης είναι να οργανώνει και να διαχειρίζεται βασικές πληροφορίες που χρειάζονται για τη λειτουργία ενός νοσοκομείου.

Η βάση υλοποιήθηκε έτσι ώστε να καλύπτει τόσο την αποθήκευση των δεδομένων όσο και την εκτέλεση σύνθετων SQL ερωτημάτων πάνω σε αυτά. Επιπλέον, δόθηκε έμφαση στο να συνδέονται σωστά οι πίνακες μεταξύ τους και να υπάρχουν βασικοί περιορισμοί που αποτρέπουν την εισαγωγή λανθασμένων δεδομένων.

Στο πλαίσιο της εργασίας δημιουργήθηκαν τα διαγράμματα (ER diagram & relational schema), τα SQL scripts για τη δημιουργία και τη φόρτωση της βάσης, καθώς και τα ζητούμενα queries. Τέλος, υλοποιήθηκε μια απλή web εφαρμογή, μέσω της οποίας ο χρήστης μπορεί να βλέπει βασικά δεδομένα της βάσης και να εκτελεί τα ζητούμενα queries πιο εύκολα.

2. Περιβάλλον υλοποίησης

Ο σχεδιασμός της βάσης ξεκίνησε από το ER διάγραμμα, το οποίο δημιουργήθηκε στο draw.io και έπειτα ακολούθησε η υλοποίηση του σχεσιακού διαγράμματος στο dbdiagram.io.

Η βάση υλοποιήθηκε σε MariaDB. Το αρχείο install.sql χρησιμοποιείται για τη συνολική εγκατάσταση της βάσης και χωρίζεται ουσιαστικά σε δύο μέρη:

- το ddl.sql, στο οποίο ορίζονται οι πίνακες και τα indexes
- το procedures.sql, στο οποίο περιλαμβάνονται functions και triggers.

Τα δεδομένα αναφοράς, όπως οι κωδικοί ΚΕΝ, οι διαγνώσεις ICD-10, οι εργαστηριακές εξετάσεις, οι ιατρικές πράξεις και τα φάρμακα, προέρχονται από αρχεία του Υπουργείου Υγείας και εισάγονται στη βάση δεδομένων από τα αντίστοιχα CSV αρχεία με χρήση Python. Όλα τα υπόλοιπα δεδομένα δημιουργούνται και φορτώνονται με χρήση της βιβλιοθήκης Faker της Python.

Όσον αφορά το φωτογραφικό υλικό, χρησιμοποιήθηκαν εικόνες νοσοκομειακού προσωπικού και χώρων από το Pexels, μια δωρεάν πλατφόρμα φωτογραφιών με περιεχόμενο διαθέσιμο για ελεύθερη χρήση (copyright-free). Η επιλογή και η λήψη των εικόνων έγιναν με Python script.

Τέλος, η υλοποίηση της web εφαρμογής έγινε με Flask, χρησιμοποιώντας HTML, CSS και Jinja2 templates.

3. Παραδοχές και σχεδιαστικές επιλογές

α) Η ηλικία των ασθενών δεν αποθηκεύεται απευθείας στη βάση, αλλά υπολογίζεται κάθε φορά με βάση την ημερομηνία γέννησης.

β) Για το προσωπικό χρησιμοποιείται ένας κοινός πίνακας Staff, στον οποίο αποθηκεύονται τα βασικά στοιχεία όλων των εργαζομένων. Οι γιατροί, οι νοσηλευτές και το διοικητικό προσωπικό έχουν επιπλέον ξεχωριστούς πίνακες όπου αποθηκεύονται τα στοιχεία που αφορούν μόνο τη δική τους κατηγορία.

γ) Κάθε γιατρός μπορεί να ανήκει σε περισσότερα από ένα τμήματα, καθώς αυτό αντιστοιχεί καλύτερα στη λειτουργία ενός νοσοκομείου.

δ) Κάθε ασθενής μπορεί να έχει περισσότερες από μία νοσηλείες στο ιστορικό του, αλλά θεωρήθηκε ότι δεν μπορεί να έχει παραπάνω από μία ενεργή νοσηλεία την ίδια χρονική στιγμή.

ε) Οι διαγνώσεις καταγράφονται με κωδικούς ICD-10, ενώ η κοστολόγηση των νοσηλειών βασίζεται στους κωδικούς ΚΕΝ. Για κάθε ΚΕΝ αποθηκεύεται το βασικό κόστος και η μέση διάρκεια νοσηλείας, ώστε να μπορεί να υπολογιστεί πρόσθετη χρέωση όταν η παραμονή του ασθενή την ξεπερνά.

στ) Οι κανόνες βαρδιών επιβάλλονται αυτόματα μέσω triggers: κάθε βάρδια χρειάζεται τουλάχιστον 3 γιατρούς, 6 νοσηλευτές και 2 διοικητικούς υπαλλήλους. Οι ειδικευόμενοι δεν μπορούν να μπουν σε βάρδια χωρίς την παρουσία ανώτερου γιατρού. Επίσης, για κάθε μέλος προσωπικού ισχύουν μηνιαία όρια βαρδιών, ελάχιστη ανάπαυση 8 ωρών μεταξύ βαρδιών και περιορισμός σε έως 3 συνεχόμενες νυχτερινές βάρδιες.

ζ) Τα στοιχεία των οικείων προσώπων για περίπτωση ανάγκης αποθηκεύονται σε ξεχωριστό πίνακα, καθώς ένας ασθενής μπορεί να έχει περισσότερα από ένα άτομα επικοινωνίας.

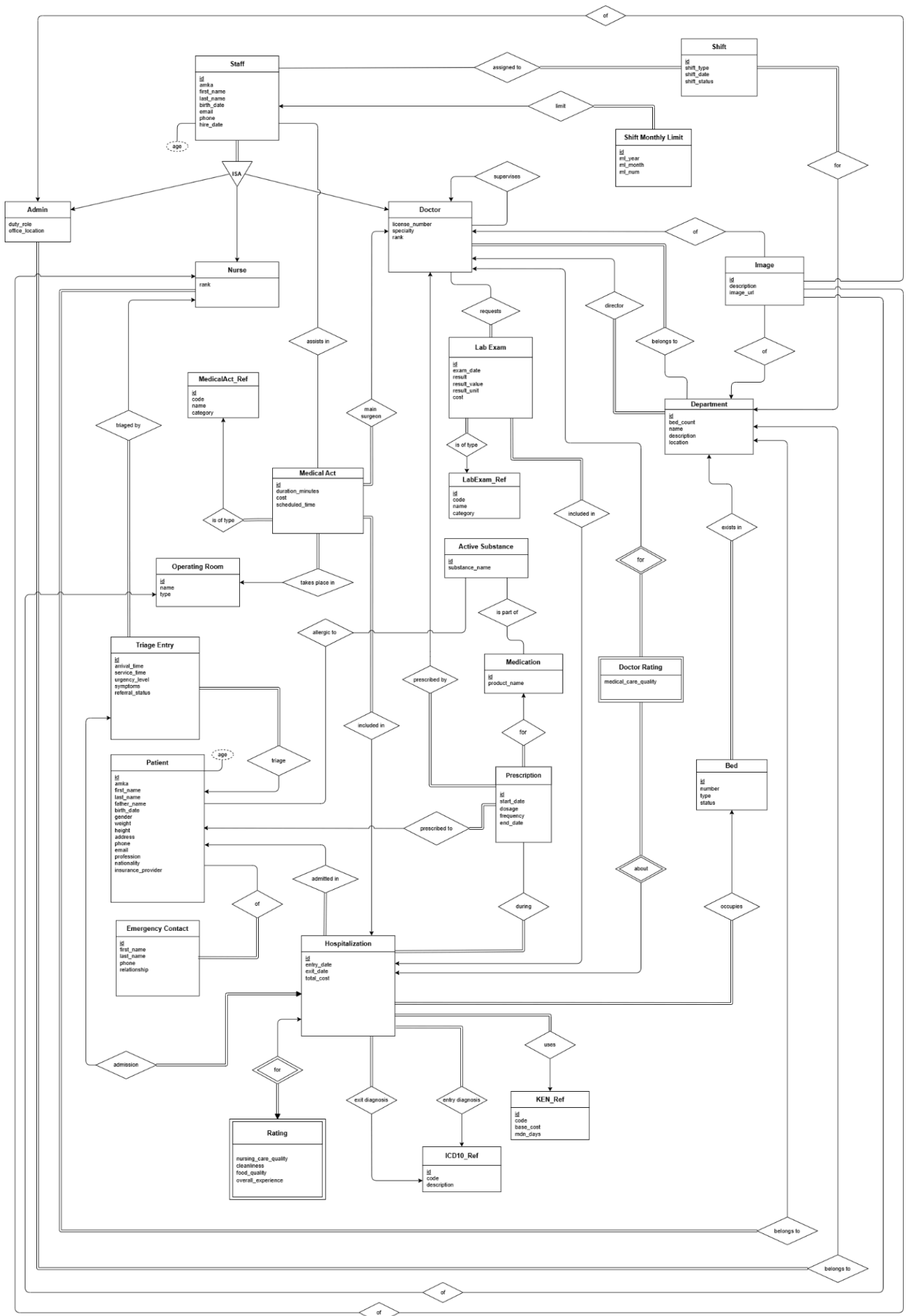
η) Οι ιατρικές πράξεις και οι εργαστηριακές εξετάσεις ταξινομούνται σύμφωνα με την κατηγοριοποίηση του Υπουργείου Υγείας. Οι ιατρικές πράξεις βασίζονται στις κατηγορίες Α και Β, ενώ οι εργαστηριακές εξετάσεις στις κατηγορίες Γ, Δ και Ε.

θ) Η διαδικασία triage γίνεται μόνο από νοσηλευτές που ανήκουν στο Τμήμα Επειγόντων. Η διαδικασία αυτή θεωρήθηκε ανεξάρτητη από το κανονικό πρόγραμμα βαρδιών.

ι) Στις ιατρικές πράξεις μπορούν να συμμετέχουν μόνο γιατροί και νοσηλευτές, καθώς το διοικητικό προσωπικό δεν έχει ιατρικό ή νοσηλευτικό ρόλο. Ο περιορισμός αυτός επιβάλλεται μέσω triggers.

4. Διάγραμμα ER

Το διάγραμμα ER αποτέλεσε το πρώτο στάδιο σχεδιασμού της βάσης μας. Σε αυτό αποτυπώνονται οι οντότητες που προκύπτουν από την εκφώνηση, καθώς και οι μεταξύ τους σχέσεις.



- Cardinalities:

Αξίζει να σημειωθεί ότι στο διάγραμμα ER αποτυπώνονται αναλυτικά οι πληθικότητες όλων των σχέσεων, σύμφωνα με τη σημειογραφία του βιβλίου «Συστήματα Βάσεων Δεδομένων» των Abraham Silberschatz, Henry F. Korth και S. Sudarshan. Συγκεκριμένα, το κατευθυνόμενο βέλος προς μια οντότητα δηλώνει την πλευρά του «ένα», ενώ η πλευρά χωρίς βέλος αντιστοιχεί στο «πολλά». Με αυτόν τον τρόπο φαίνεται αν μια σχέση είναι “Ένα-προς-ένα”, “Ένα-προς-πολλά”, “Πολλά-προς-ένα” ή “Πολλά-προς-πολλά”. Επίσης, η διπλή γραμμή δηλώνει ολική συμμετοχή μιας οντότητας σε μια σχέση (total participation), ενώ η απλή γραμμή δηλώνει μερική συμμετοχή (partial participation).

- Σχέση ISA:

Όσον αφορά τη σχέση ISA, αυτή χρησιμοποιείται για την εξειδίκευση της οντότητας Staff στις επιμέρους κατηγορίες Doctor, Nurse και Admin. Η εξειδίκευση αυτή θεωρείται ασυνεχής, καθώς κάθε μέλος του προσωπικού ανήκει σε μία μόνο από αυτές τις κατηγορίες. Με αυτόν τον τρόπο τα κοινά στοιχεία του προσωπικού αποθηκεύονται στον πίνακα Staff, ενώ τα ειδικά στοιχεία κάθε κατηγορίας αποθηκεύονται ξεχωριστά.

- Weak entities:

Ως ασθενείς οντότητες μοντελοποιούμε το Rating και το DoctorRating, καθώς η ύπαρξή τους εξαρτάται από συγκεκριμένη νοσηλεία. Το Rating αφορά την αξιολόγηση μιας νοσηλείας από τον ασθενή και έχει ως primary key το hospitalization_id, αφού για κάθε νοσηλεία θεωρούμε λογικό να υπάρχει το πολύ μία αξιολόγηση. Αντίστοιχα, το DoctorRating αφορά την αξιολόγηση συγκεκριμένου γιατρού που έχει συνταγογραφήσει στο πλαίσιο συγκεκριμένης νοσηλείας και έχει σύνθετο primary key το (hospitalization_id, doctor_id), ώστε για κάθε ζεύγος νοσηλείας και γιατρού να υπάρχει το πολύ μία αξιολόγηση. Ο έλεγχος ότι ο γιατρός που αξιολογείται έχει πράγματι συνταγογραφήσει στη συγκεκριμένη νοσηλεία γίνεται μέσω trigger και αναλύεται στη συνέχεια.

5. Σχεσιακό διάγραμμα

Στη συνέχεια, το διάγραμμα ER μετατράπηκε σε σχεσιακό διάγραμμα ώστε να προκύψουν οι τελικοί πίνακες της βάσης. Κάθε οντότητα αποδόθηκε ως πίνακας με τα αντίστοιχα πεδία της, ενώ οι σχέσεις μεταξύ των οντοτήτων υλοποιήθηκαν με foreign keys και, όπου χρειαζόταν, με junction tables.



Ιδιαίτερη σημασία έχουν οι σχέσεις “Πολλά-προς-πολλά” (many-to-many), οι οποίες δεν μπορούν να αποδοθούν απευθείας με foreign key. Για αυτές δημιουργήθηκαν τα εξής junction tables:

- DoctorDepartment για τη σχέση ανάμεσα σε γιατρούς και τμήματα, αφού ένας γιατρός μπορεί να ανήκει σε περισσότερα από ένα τμήματα και κάθε τμήμα μπορεί να έχει πολλούς γιατρούς.
- Staff_Shift για τη συσχέτιση του προσωπικού με τις βάρδιες, καθώς ένα μέλος προσωπικού μπορεί να συμμετέχει σε πολλές βάρδιες και κάθε βάρδια περιλαμβάνει πολλά μέλη προσωπικού.
- Medical_Act_Assistant για τους βοηθούς που συμμετέχουν σε μια ιατρική πράξη, αφού μια πράξη μπορεί να έχει πολλούς βοηθούς και ένα μέλος προσωπικού μπορεί να συμμετέχει ως βοηθός σε πολλές πράξεις.
- MedicationSubstance για τη σχέση ανάμεσα σε φάρμακα και δραστικές ουσίες, καθώς ένα φάρμακο μπορεί να περιέχει πολλές δραστικές ουσίες και μία δραστική ουσία μπορεί να υπάρχει σε πολλά φάρμακα.
- Allergy για τη σχέση ανάμεσα σε ασθενείς και δραστικές ουσίες στις οποίες έχουν αλλεργία.

Σε αυτούς τους πίνακες χρησιμοποιείται σύνθετο primary key, το οποίο αποτελείται από τα αντίστοιχα foreign keys προς τις δύο οντότητες που συσχετίζονται.

❖ Σημείωση:

Υπάρχει ένας περιορισμός στην οπτική αναπαράσταση του dbdiagram.io. Συγκεκριμένα, ο πίνακας Hospitalization έχει δύο foreign keys προς τον πίνακα ICD10_Ref, ένα για τη διάγνωση εισαγωγής και ένα για τη διάγνωση εξόδου. Παρότι ένας κωδικός ICD-10 μπορεί να χρησιμοποιείται σε πολλές νοσηλείες, η πλατφόρμα εμφανίζει λανθασμένα την πληθικότητα σε αυτό το σημείο ως 0..1 (αντί για 0..*) λόγω των παράλληλων συνδέσεων. Το πρόβλημα αφορά μόνο την οπτική απεικόνιση του dbdiagram.io και δεν σχετίζεται με την ορθότητα του κώδικα και των foreign keys.

6. Υλοποίηση Βάσης

Η υλοποίηση της βάσης γίνεται μέσω του αρχείου install.sql, το οποίο συγκεντρώνει τα απαραίτητα SQL scripts. Συγκεκριμένα, το ddl.sql περιλαμβάνει τη δημιουργία των πινάκων και των indexes, ενώ το procedures.sql περιλαμβάνει τις functions και τα triggers που χρησιμοποιούνται στη βάση.

Στο ddl.sql ορίζονται τα primary keys, τα foreign keys και οι βασικοί περιορισμοί των πινάκων. Τα foreign keys χρησιμοποιούνται για να αποτυπωθούν οι σχέσεις που προέκυψαν από το σχεσιακό διάγραμμα, ενώ τα CHECK constraints περιορίζουν τις επιτρεπτές τιμές σε συγκεκριμένα πεδία, όπως τύπος προσωπικού, βαθμίδα γιατρού, τύπος βάρδιας, επίπεδο triage και τιμές αξιολόγησης.

Για πιο σύνθετους κανόνες, που δεν μπορούν να καλυφθούν εύκολα μόνο με απλούς περιορισμούς, χρησιμοποιούνται triggers και functions. Τέτοιοι κανόνες αφορούν, για

παράδειγμα, την εποπτεία των γιατρών, τις βάρδιες του προσωπικού, τις αλλεργίες σε δραστικές ουσίες, τη διαθεσιμότητα κλινών, τις συγκρούσεις σε ιατρικές πράξεις και τους ελέγχους στις αξιολογήσεις.

Το πλήρες DDL script, με τους ορισμούς των πινάκων, των περιορισμών και των indexes, παρατίθεται στο Παράρτημα Α.

7. Περιορισμοί & Triggers

Οι περιορισμοί ακεραιότητας (δηλαδή τα primary keys) και οι περιορισμοί αναφορικής ακεραιότητας (τα foreign keys), φαίνονται αναλυτικά στο σχεσιακό διάγραμμα και υλοποιούνται στο αρχείο ddl.sql. Τα primary keys εξασφαλίζουν τη μοναδικότητα των εγγραφών, ενώ τα foreign keys διατηρούν τις σωστές σχέσεις μεταξύ των πινάκων.

Για τους περιορισμούς ακεραιότητας πεδίου τιμών χρησιμοποιήθηκαν CHECK constraints, ενώ για πιο σύνθετους περιορισμούς οριζόμενους από τον χρήστη χρησιμοποιήθηκαν triggers και βοηθητικές συναρτήσεις.

Στη συνέχεια παρουσιάζονται αναλυτικά οι περιορισμοί που χρησιμοποιήθηκαν.

CHECK Constraints:

- **chk_staff_type**
Ο τύπος προσωπικού στον πίνακα Staff μπορεί να είναι μόνο doctor, nurse ή admin.
- **chk_gender**
Το φύλο του ασθενή στον πίνακα Patients μπορεί να είναι μόνο Male, Female ή Other.
- **chk_doctor_rank**
Η βαθμίδα του γιατρού στον πίνακα Doctors περιορίζεται στις τιμές Ειδικευόμενος, Επιμελητής Β΄, Επιμελητής Α΄ και Διευθυντής.
- **chk_supervisor**
Ο ειδικευόμενος γιατρός πρέπει υποχρεωτικά να έχει επόπτη, ενώ ο Διευθυντής δεν επιτρέπεται να έχει επόπτη.
- **chk_nurse_rank**
Η βαθμίδα του νοσηλεύτη στον πίνακα Nurses μπορεί να είναι μόνο Βοηθός Νοσηλεύτη, Νοσηλεύτης ή Προϊστάμενος.
- **chk_bed_status**
Η κατάσταση μιας κλίνης στον πίνακα Beds μπορεί να είναι μόνο Διαθέσιμη, Κατειλημμένη ή Υπό συντήρηση.
- **CHECK (urgency_level BETWEEN 1 AND 5)**
Το επίπεδο επείγοντος στο triage πρέπει να είναι από 1 έως 5.
- **chk_referral**
Το αποτέλεσμα του triage μπορεί να είναι είτε Exit, δηλαδή ο ασθενής αποχωρεί, είτε Hospitalization, δηλαδή παραπέμπεται για νοσηλεία.

- **chk_shifttype**
Ο τύπος βάρδιας στον πίνακα Shifts μπορεί να είναι μόνο Morning, Afternoon ή Night.
- **chk_shiftstatus**
Η κατάσταση μιας βάρδιας μπορεί να είναι μόνο scheduled, ongoing, completed ή cancelled.
- **CHECK (ml_month BETWEEN 1 AND 12)**
Ο μήνας στον πίνακα Shift_Monthly_Limits πρέπει να έχει τιμή από 1 έως 12.
- **chk_lab_ref_category**
Οι εργαστηριακές εξετάσεις στον πίνακα LabExam_Ref περιορίζονται στις κατηγορίες Γ, Δ και Ε της ελληνικής ονοματολογίας ιατρικών πράξεων.
- **chk_act_ref_category**
Οι ιατρικές πράξεις στον πίνακα MedicalAct_Ref περιορίζονται στις κατηγορίες Α και Β της ελληνικής ονοματολογίας ιατρικών πράξεων.
- **CHECK στις στήλες αξιολόγησης του πίνακα Hospitalization_Ratings**
Οι τιμές nursing_care_quality, cleanliness, food_quality και overall_experience πρέπει να είναι από 1 έως 5.
- **CHECK στη στήλη medical_care_quality του πίνακα Doctor_Ratings**
Η αξιολόγηση της ιατρικής φροντίδας πρέπει να είναι από 1 έως 5.
- **chk_image_entity**
Κάθε εικόνα στον πίνακα Images πρέπει να αντιστοιχεί σε ακριβώς μία οντότητα, δηλαδή είτε σε γιατρό, είτε σε νοσηλεύτη, είτε σε διοικητικό υπάλληλο, είτε σε τμήμα, είτε σε χώρο επέμβασης.

UNIQUE constraints:

Χρησιμοποιούνται ώστε να αποφεύγονται διπλές εγγραφές σε περιπτώσεις όπου ένας συνδυασμός πεδίων πρέπει να εμφανίζεται μόνο μία φορά. Συγκεκριμένα:

- κάθε εγγραφή triage μπορεί να αντιστοιχεί σε μία μόνο νοσηλεία
- κάθε συνταγογράφηση είναι μοναδική ως προς τον συνδυασμό γιατρού, ασθενή, φαρμάκου και ημερομηνίας έναρξης
- κάθε βάρδια είναι μοναδική για συγκεκριμένο τμήμα, ημερομηνία και τύπο βάρδιας
- κάθε μέλος προσωπικού έχει μία μόνο εγγραφή μηνιαίου μετρητή βαρδιών ανά μήνα και έτος.

Βοηθητικές συναρτήσεις:

- **has_cycle**
Ελέγχει αν δημιουργείται κυκλική αλυσίδα εποπτείας μεταξύ γιατρών.
- **check_min_staff_per_shift**
Ελέγχει αν μια βάρδια έχει τον ελάχιστο απαιτούμενο αριθμό γιατρών, νοσηλευτών και διοικητικών.
- **check_resident_supervisor**
Ελέγχει αν, όταν υπάρχει ειδικευόμενος σε βάρδια, υπάρχει μαζί του Επιμελητής Α΄ ή Διευθυντής.
- **check_monthly_limit**
Ελέγχει αν ένα μέλος προσωπικού έχει ξεπεράσει το μηνιαίο όριο βαρδιών.

- **check_consecutive_nights**
Ελέγχει αν ξεπερνιέται το όριο των 3 συνεχόμενων νυχτερινών βαρδιών.
- **get_shift_start**
Υπολογίζει την ώρα έναρξης μιας βάρδιας ανάλογα με το αν είναι πρωινή, απογευματινή ή νυχτερινή.
- **get_shift_end**
Υπολογίζει την ώρα λήξης μιας βάρδιας.
- **check_rest_period**
Ελέγχει αν υπάρχει τουλάχιστον 8ωρη ανάπαυση μεταξύ δύο βαρδιών του ίδιου εργαζομένου.

Triggers:

Ενεργοποιούνται αυτόματα σε ενέργειες όπως εισαγωγή ή ενημέρωση εγγραφών, εμποδίζοντας λανθασμένα δεδομένα να περάσουν στη βάση.

- **trg_supervisor_deleted**
Εκτελείται πριν από ενημέρωση στον πίνακα Doctors. Εμποδίζει έναν ειδικευόμενο γιατρό να μείνει χωρίς επόπτη, αν επιχειρηθεί να γίνει το supervisor_id του ίσο με NULL.
- **trg_doctor_insert_no_cycle**
Εκτελείται πριν από την εισαγωγή νέου γιατρού. Ελέγχει, με τη βοήθεια της function has_cycle, ότι η ανάθεση επόπτη δεν δημιουργεί κυκλική αλυσίδα εποπτείας.
- **trg_doctor_update_no_cycle**
Εκτελείται πριν από αλλαγή στοιχείων γιατρού. Αν αλλάξει ο επόπτης, ελέγχει ξανά ότι δεν δημιουργείται κύκλος στη σχέση εποπτείας.
- **trg_check_allergy_before_prescription**
Εκτελείται πριν από κάθε νέα συνταγογράφηση. Ελέγχει αν ο ασθενής έχει δηλωμένη αλλεργία σε κάποια δραστική ουσία του φαρμάκου. Αν βρεθεί τέτοια αλλεργία, η συνταγογράφηση απορρίπτεται.
- **trg_staff_shifts_before_insert**
Εκτελείται πριν προστεθεί προσωπικό σε βάρδια. Ελέγχει ότι:
 1. η βάρδια δεν είναι ολοκληρωμένη ή ακυρωμένη
 2. το μέλος προσωπικού δεν έχει ξεπεράσει το μηνιαίο όριο βαρδιών
 3. αν υπάρχει ειδικευόμενος, υπάρχει και Επιμελητής Α΄ ή Διευθυντής
 4. έχει τηρηθεί το ελάχιστο διάστημα ανάπαυσης 8 ωρών
 5. δεν ξεπερνιέται το όριο των 3 συνεχόμενων νυχτερινών βαρδιών.
- **trg_check_min_staff_on_shift_start**
Εκτελείται πριν από ενημέρωση στον πίνακα Shifts. Όταν μια βάρδια αλλάζει κατάσταση σε ongoing, ελέγχει ότι υπάρχει ο ελάχιστος απαιτούμενος αριθμός προσωπικού για να ξεκινήσει η βάρδια.
- **trg_update_monthly_limits**
Εκτελείται μετά την εισαγωγή προσωπικού σε βάρδια. Ενημερώνει τον πίνακα Shift_Monthly_Limits, αυξάνοντας τον μετρητή βαρδιών του συγκεκριμένου μέλους προσωπικού για τον αντίστοιχο μήνα.
- **trg_check_staff_type_doctor_insert**
Εκτελείται πριν από εισαγωγή στον πίνακα Doctors. Ελέγχει ότι το αντίστοιχο μέλος προσωπικού στον πίνακα Staff έχει staff_type = 'doctor'.

- **trg_check_staff_type_nurse_insert**
Εκτελείται πριν από εισαγωγή στον πίνακα Nurses. Ελέγχει ότι το αντίστοιχο μέλος προσωπικού έχει staff_type = 'nurse'.
- **trg_check_staff_type_admin_insert**
Εκτελείται πριν από εισαγωγή στον πίνακα Administrative_Staff. Ελέγχει ότι το αντίστοιχο μέλος προσωπικού έχει staff_type = 'admin'.
- **trg_check_bed_availability**
Εκτελείται πριν από εισαγωγή νέας νοσηλείας. Ελέγχει ότι η κλίνη που επιλέγεται δεν χρησιμοποιείται ήδη σε άλλη ενεργή νοσηλεία (δηλαδή σε νοσηλεία χωρίς ημερομηνία εξόδου).
- **trg_bed_status_on_admission**
Εκτελείται μετά την εισαγωγή νέας νοσηλείας. Αλλάζει αυτόματα την κατάσταση της αντίστοιχης κλίνης σε Κατειλημμένη.
- **trg_bed_status_on_discharge**
Εκτελείται μετά από ενημέρωση νοσηλείας. Όταν προστεθεί ημερομηνία εξόδου, αλλάζει αυτόματα την κατάσταση της κλίνης σε Διαθέσιμη.
- **trg_check_rating_on_completed_hospitalization**
Εκτελείται πριν από εισαγωγή αξιολόγησης νοσηλείας. Επιτρέπει την αξιολόγηση μόνο αν η νοσηλεία έχει ολοκληρωθεί (δηλαδή αν υπάρχει exit_date).
- **trg_check_doctor_rating_on_completed_hospitalization**
Εκτελείται πριν από εισαγωγή αξιολόγησης γιατρού. Ελέγχει ότι η αξιολόγηση αφορά ολοκληρωμένη νοσηλεία.
- **trg_check_medical_act_conflicts**
Εκτελείται πριν από εισαγωγή νέας ιατρικής πράξης. Ελέγχει ότι:
 1. δεν υπάρχει άλλη πράξη την ίδια ώρα στον ίδιο χώρο
 2. ο κύριος γιατρός δεν συμμετέχει ήδη σε άλλη πράξη με επικαλυπτόμενο χρονικό διάστημα.
- **trg_check_assistant_conflict**
Εκτελείται πριν από εισαγωγή βοηθού σε ιατρική πράξη. Ελέγχει ότι ο βοηθός δεν συμμετέχει ταυτόχρονα σε άλλη ιατρική πράξη.
- **trg_check_doctor_rating_prescribed**
Εκτελείται πριν από εισαγωγή αξιολόγησης γιατρού. Ελέγχει ότι ο γιατρός που αξιολογείται έχει πράγματι συνταγογραφήσει φάρμακο στο πλαίσιο της συγκεκριμένης νοσηλείας.
- **trg_calculate_total_cost**
Εκτελείται πριν από ενημέρωση νοσηλείας. Όταν καταχωρείται ημερομηνία εξόδου και υπάρχει κωδικός KEN, υπολογίζει αυτόματα το συνολικό κόστος νοσηλείας. Αν η διάρκεια νοσηλείας δεν ξεπερνά τη μέση διάρκεια του KEN, το κόστος ισούται με το βασικό κόστος. Αν την ξεπερνά, προστίθεται χρέωση για τις επιπλέον ημέρες.
- **trg_check_assistant_staff_type**
Εκτελείται πριν από εισαγωγή βοηθού σε ιατρική πράξη. Ελέγχει ότι ο βοηθός είναι είτε γιατρός είτε νοσηλευτής. Έτσι αποκλείεται η συμμετοχή διοικητικού προσωπικού σε ιατρικές πράξεις.
- **trg_check_triage_nurse_department**
Εκτελείται πριν από εισαγωγή εγγραφής triage. Ελέγχει ότι ο νοσηλευτής διαλογής ανήκει στο τμήμα Επείγοντα.

- **trg_update_bed_count_on_insert**
Εκτελείται μετά την εισαγωγή νέας κλίνης. Αυξάνει αυτόματα τον αριθμό κλινών του αντίστοιχου τμήματος.
- **trg_update_bed_count_on_delete**
Εκτελείται μετά τη διαγραφή κλίνης. Μειώνει αυτόματα τον αριθμό κλινών του αντίστοιχου τμήματος.

❖ Σημείωση:

Στο αρχείο `procedures.sql` χρησιμοποιείται η εντολή `DELIMITER`, ώστε να μπορούν να οριστούν σωστά τα `triggers` και τα `functions` που περιέχουν περισσότερες από μία SQL εντολές. Με αυτόν τον τρόπο η MariaDB δεν μπερδεύει το τέλος κάθε εσωτερικής εντολής με το τέλος ολόκληρου του `trigger`.

8. Ευρετήρια (Indexes)

Τα `indexes` είναι ειδικές δομές που δημιουργούνται σε μία ή περισσότερες στήλες ενός πίνακα, με σκοπό να γίνεται πιο γρήγορη η αναζήτηση και η προσπέλαση των δεδομένων. Είναι ιδιαίτερα χρήσιμα σε ερωτήματα που χρησιμοποιούν `WHERE`, `JOIN`, `GROUP BY` και `ORDER BY`, γιατί μειώνουν την ανάγκη για πλήρη σάρωση ολόκληρων πινάκων.

Στη δική μας βάση, ορισμένα `indexes` δημιουργούνται ήδη αυτόματα από τη MariaDB λόγω των `primary keys` και των `UNIQUE constraints`. Παρ' όλα αυτά, στο αρχείο `ddl.sql` ορίστηκαν και επιπλέον `indexes` σε πεδία που χρησιμοποιούνται συχνά στα ζητούμενα `queries` και στα `joins` μεταξύ των πινάκων. Αυτά είναι τα εξής:

- **CREATE INDEX idx_staff_name ON Staff(last_name, first_name);**

Χρησιμοποιείται για αναζητήσεις προσωπικού με βάση το επώνυμο και το όνομα. Είναι χρήσιμο σε περιπτώσεις όπου θέλουμε να εμφανίσουμε ή να φιλτράρουμε μέλη του προσωπικού με βάση τα προσωπικά τους στοιχεία, χωρίς να χρειάζεται να σαρωθεί όλος ο πίνακας `Staff`.

- **CREATE INDEX idx_staff_type ON Staff(staff_type);**

Βοηθά στη διάκριση του προσωπικού ανά κατηγορία, δηλαδή σε γιατρούς, νοσηλευτές και διοικητικούς υπαλλήλους. Είναι χρήσιμο σε `queries` και ελέγχους που χρειάζονται ομαδοποίηση ή φιλτράρισμα με βάση τον τύπο προσωπικού.

- **CREATE INDEX idx_patients_name ON Patients(last_name, first_name);**

Επιταχύνει την αναζήτηση ασθενών με βάση το επώνυμο και το όνομα. Χρησιμοποιείται κυρίως όταν χρειάζεται να εντοπιστεί συγκεκριμένος ασθενής ή να εμφανιστεί λίστα ασθενών φιλτραρισμένη με βάση τα στοιχεία τους.

- **CREATE INDEX idx_patients_insurance ON Patients(insurance_provider);**

Είναι χρήσιμο για ερωτήματα που ομαδοποιούν ασθενείς με βάση τον ασφαλιστικό φορέα. Για παράδειγμα, στο ερώτημα εσόδων ανά τμήμα και ανά έτος χρειάζεται και κατανομή νοσηλείων ανά ασφαλιστικό φορέα, οπότε το index βοηθά στην πιο γρήγορη πρόσβαση στη στήλη insurance_provider.

- **CREATE INDEX idx_doctors_specialty ON Doctors(specialty);**

Χρησιμοποιείται σε ερωτήματα όπου ζητούνται γιατροί συγκεκριμένης ειδικότητας. Για παράδειγμα, στο Q2 χρειάζεται αναζήτηση γιατρών με βάση συγκεκριμένη ειδικότητα, άρα το index αποφεύγει την πλήρη σάρωση του πίνακα Doctors.

- **CREATE INDEX idx_doctors_rank ON Doctors(rank);**

Βοηθά σε ελέγχους και queries που βασίζονται στη βαθμίδα του γιατρού. Είναι χρήσιμο για τον εντοπισμό ειδικευόμενων, Επιμελητών Α΄ και Διευθυντών, ειδικά στους κανόνες βαρδιών όπου ένας ειδικευόμενος πρέπει να βρίσκεται μαζί με ανώτερο γιατρό.

- **CREATE INDEX idx_doctors_supervisor ON Doctors(supervisor_id);**

Χρησιμοποιείται για τη σχέση εποπτείας μεταξύ γιατρών. Βοηθά στην πιο γρήγορη εύρεση γιατρών με συγκεκριμένο επόπτη και είναι χρήσιμο στο Q13, όπου ζητείται η ιεραρχία εποπτείας κάθε γιατρού.

- **CREATE INDEX idx_departments_head ON Departments(head_doctor_id);**

Βοηθάει στη σύνδεση κάθε τμήματος με τον διευθυντή του. Είναι χρήσιμο σε joins μεταξύ Departments και Doctors όταν χρειάζεται να εμφανιστεί ο επικεφαλής γιατρός ενός τμήματος.

- **CREATE INDEX idx_beds_status ON Beds(status);**

Χρησιμοποιείται για την αναζήτηση κλινών με βάση την κατάστασή τους (διαθέσιμες, κατειλημμένες ή υπό συντήρηση). Είναι χρήσιμο για ελέγχους διαθεσιμότητας κλινών κατά την εισαγωγή νέας νοσηλείας.

- **CREATE INDEX idx_beds_department ON Beds(department_id);**

Βοηθάει στην αναζήτηση κλινών ανά τμήμα. Είναι χρήσιμο όταν χρειάζεται να βρεθούν οι κλίνες ενός συγκεκριμένου τμήματος ή να ενημερωθεί ο αριθμός κλινών του τμήματος.

- **CREATE INDEX idx_triage_patient ON Triage_Entries(patient_id);**

Επιταχύνει την αναζήτηση εγγράφων triage για συγκεκριμένο ασθενή. Είναι χρήσιμο για τη σύνδεση του ιστορικού ενός ασθενή με το triage entry που μπορεί να οδήγησε σε νοσηλεία.

- **CREATE INDEX idx_triage_urgency ON Triage_Entries(urgency_level);**

Χρησιμοποιείται σε ερωτήματα που ομαδοποιούν τα triage entries με βάση το επίπεδο επείγοντος. Είναι ιδιαίτερα χρήσιμο στο Q15, όπου ζητείται η κατανομή αυτών των περιστατικών ανά επίπεδο επείγοντος.

- **CREATE INDEX idx_triage_arrival ON Triage_Entries(arrival_time);**

Βοηθάει στην ταξινόμηση των triage entries με βάση τον χρόνο άφιξης. Είναι χρήσιμο επειδή για το ίδιο επίπεδο επείγοντος η εξυπηρέτηση γίνεται με σειρά προτεραιότητας FIFO.

- **CREATE INDEX idx_hosp_patient ON Hospitalizations(patient_id);**

Το index αυτό είναι πολύ σημαντικό, επειδή ο πίνακας Hospitalizations χρησιμοποιείται σε πολλά ερωτήματα. Συγκεκριμένα, βοηθά στην αναζήτηση όλων των νοσηλειών ενός συγκεκριμένου ασθενή, όπως στο Q6 όπου ζητείται το ιστορικό νοσηλειών ενός ασθενή.

- **CREATE INDEX idx_hosp_department ON Hospitalizations(department_id);**

Βοηθάει σε ερωτήματα που εξετάζουν τις νοσηλείες ανά τμήμα. Είναι χρήσιμο στο Q1 για τα έσοδα ανά τμήμα, αλλά και στο Q3 όπου ζητούνται ασθενείς που νοσηλεύτηκαν πολλές φορές στο ίδιο τμήμα.

- **CREATE INDEX idx_hosp_dates ON Hospitalizations(entry_date, exit_date);**

Χρησιμοποιείται για ερωτήματα που ομαδοποιούν νοσηλείες με βάση χρονικά διαστήματα. Είναι χρήσιμο σε queries ανά έτος, σε υπολογισμούς διάρκειας νοσηλείας και σε ελέγχους για ενεργές ή ολοκληρωμένες νοσηλείες.

- **CREATE INDEX idx_hosp_ken ON Hospitalizations(ken_code);**

Το index αυτό βοηθάει στη σύνδεση των νοσηλειών με τους κωδικούς KEN. Είναι χρήσιμο στο Q1, όπου γίνεται ανάλυση εσόδων ανά KEN, καθώς και στον υπολογισμό του κόστους νοσηλείας με βάση το βασικό κόστος και τη μέση διάρκεια νοσηλείας.

- **CREATE INDEX idx_hosp_icd10 ON Hospitalizations(icd10_entry_code);**

Χρησιμοποιείται για αναζητήσεις και ομαδοποιήσεις με βάση τη διάγνωση εισαγωγής ICD-10. Είναι χρήσιμο κυρίως στο Q14, όπου ζητούνται κατηγορίες ICD-10 με ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη.

- **CREATE INDEX idx_shifts_date ON Shifts(shift_date);**

Βοηθάει στην αναζήτηση βαρδιών συγκεκριμένης ημερομηνίας. Είναι χρήσιμο σε ερωτήματα που αφορούν το πρόγραμμα εφημεριών, όπως στο Q8 όπου ζητείται το προσωπικό που δεν έχει βάρδια σε συγκεκριμένη ημερομηνία και τμήμα.

- **CREATE INDEX idx_shifts_type ON Shifts(shift_type);**

Επιταχύνει την αναζήτηση βαρδιών με βάση τον τύπο τους (πρωινή, απογευματινή, νυχτερινή). Επίσης, είναι χρήσιμο στους ελέγχους για συνεχόμενες νυχτερινές βάρδιες.

- **CREATE INDEX idx_shifts_department ON Shifts(department_id, shift_date);**

Βοηθάει σε ερωτήματα που αναζητούν βάρδιες συγκεκριμένου τμήματος σε συγκεκριμένο χρονικό διάστημα. Είναι χρήσιμο στο Q8 και στο Q12, όπου οι βάρδιες εξετάζονται αντίστοιχα ανά τμήμα και ημερομηνία ή ανά τμήμα και εβδομάδα.

- **CREATE INDEX idx_staff_shifts_staff ON Staff_Shifts(staff_id);**

Το index αυτό βοηθάει στην εύρεση όλων των βαρδιών ενός συγκεκριμένου μέλους προσωπικού. Είναι χρήσιμο για τους ελέγχους μηνιαίων ορίων, ελάχιστης ανάπαυσης και συνεχόμενων νυχτερινών βαρδιών.

- **CREATE INDEX idx_staff_shifts_shift ON Staff_Shifts(shift_id);**

Βοηθάει στην εύρεση όλου του προσωπικού που συμμετέχει σε μια συγκεκριμένη βάρδια. Είναι χρήσιμο στους ελέγχους ελάχιστου προσωπικού ανά βάρδια, αλλά και σε queries που χρειάζονται σύνδεση βαρδιών με προσωπικό.

- **CREATE INDEX idx_sml_staff_date ON Shift_Monthly_Limits(staff_id, ml_year, ml_month);**

Χρησιμοποιείται για τον γρήγορο έλεγχο των μηνιαίων ορίων των βαρδιών ανά εργαζόμενο, έτος και μήνα. Είναι χρήσιμο στο trigger που ενημερώνει και ελέγχει πόσες βάρδιες έχει ήδη κάνει ένα μέλος προσωπικού μέσα στον μήνα.

- **CREATE INDEX idx_presc_doctor ON Prescriptions(doctor_id);**

Βοηθάει στην αναζήτηση συνταγογραφήσεων ανά γιατρό. Είναι χρήσιμο στο Q4, όπου για συγκεκριμένο γιατρό ζητούνται οι αξιολογήσεις των ασθενών του, αλλά και στον έλεγχο ότι ένας γιατρός που αξιολογείται έχει πράγματι συνταγογραφήσει στη συγκεκριμένη νοσηλεία.

- **CREATE INDEX idx_presc_patient ON Prescriptions(patient_id);**

Το συγκεκριμένο index βοηθάει στην αναζήτηση συνταγογραφήσεων ανά ασθενή. Είναι χρήσιμο σε ερωτήματα που εξετάζουν το ιστορικό αγωγής ενός ασθενή ή συνδέουν φάρμακα με συγκεκριμένες νοσηλείες.

- **CREATE INDEX idx_presc_medication ON Prescriptions(medication_id);**

Χρησιμοποιείται για την αναζήτηση συνταγογραφήσεων ανά φάρμακο. Είναι χρήσιμο σε ερωτήματα που εξετάζουν τη χρήση φαρμάκων και τη σύνδεσή τους με δραστικές ουσίες.

- **CREATE INDEX idx_presc_hospitalization ON Prescriptions(hospitalization_id);**

Βοηθάει στη σύνδεση των συνταγογραφήσεων με τη νοσηλεία στην οποία ανήκουν. Είναι χρήσιμο σε queries που εξετάζουν τη φαρμακευτική αγωγή στο πλαίσιο μιας συγκεκριμένης νοσηλείας, όπως στο Q10 όπου αναζητούνται συνδυασμοί δραστικών ουσιών που συνταγογραφήθηκαν στον ίδιο ασθενή κατά την ίδια νοσηλεία.

- **CREATE INDEX idx_presc_dates ON Prescriptions(start_date, end_date);**

Χρησιμοποιείται για αναζητήσεις που βασίζονται στο χρονικό διάστημα της συνταγογράφησης. Είναι χρήσιμο όταν χρειάζεται να ελεγχθεί αν δύο φάρμακα δόθηκαν ταυτόχρονα ή αν μια αγωγή ήταν ενεργή σε συγκεκριμένη περίοδο.

- **CREATE INDEX idx_medsbst_medication**

ON Medication_Substances(medication_id);

Βοηθάει στη σύνδεση ενός φαρμάκου με τις δραστικές ουσίες που περιέχει. Είναι απαραίτητο για τον έλεγχο αλλεργιών πριν από συνταγογράφηση, αλλά και για queries που αναλύουν φάρμακα με βάση τις δραστικές τους ουσίες.

- **CREATE INDEX idx_medsbst_substance**

ON Medication_Substances(substance_id);

Επιταχύνει την αναζήτηση φαρμάκων που περιέχουν συγκεκριμένη δραστική ουσία. Είναι χρήσιμο στο Q7, όπου ζητείται για κάθε δραστική ουσία ο αριθμός φαρμάκων που την περιέχουν.

- **CREATE INDEX idx_allergies_patient ON Patient_Allergies(patient_id);**

Βοηθάει στον γρήγορο έλεγχο των αλλεργιών ενός συγκεκριμένου ασθενή. Είναι χρήσιμο στο trigger που απορρίπτει μια συνταγογράφηση αν ο ασθενής έχει αλλεργία σε δραστική ουσία του φαρμάκου.

- **CREATE INDEX idx_allergies_substance ON Patient_Allergies(substance_id);**

Χρησιμοποιείται για την εύρεση όλων των ασθενών που έχουν αλλεργία σε συγκεκριμένη δραστική ουσία. Είναι χρήσιμο στο Q7, όπου υπολογίζεται ο αριθμός αλλεργικών ασθενών ανά δραστική ουσία.

- **CREATE INDEX idx_medacts_doctor ON Medical_Acts(main_doctor_id);**

Βοηθάει στην αναζήτηση ιατρικών πράξεων ανά κύριο γιατρό. Είναι χρήσιμο στο Q5 και στο Q11, όπου ζητούνται γιατροί με βάση τον αριθμό επεμβάσεων που έχουν εκτελέσει ως κύριοι χειρουργοί.

- **CREATE INDEX idx_medacts_room ON Medical_Acts(room_id);**

Βοηθάει στην αναζήτηση ιατρικών πράξεων ανά χώρο επέμβασης. Είναι χρήσιμο στους ελέγχους σύγκρουσης, ώστε να μη γίνονται δύο ιατρικές πράξεις ταυτόχρονα στον ίδιο χώρο.

- **CREATE INDEX idx_medacts_time ON Medical_Acts(scheduled_time);**

Χρησιμοποιείται για αναζητήσεις με βάση την προγραμματισμένη ώρα μιας ιατρικής πράξης. Είναι χρήσιμο στους ελέγχους χρονικής επικάλυψης πράξεων, είτε στον ίδιο χώρο είτε με τον ίδιο γιατρό.

- **CREATE INDEX idx_labexam_date ON LabExam(exam_date);**

Βοηθάει σε αναζητήσεις εργαστηριακών εξετάσεων με βάση την ημερομηνία εξέτασης. Είναι χρήσιμο όταν χρειάζεται να φιλτραριστούν εξετάσεις συγκεκριμένης χρονικής περιόδου.

- **CREATE INDEX idx_labexam_hospitalization ON LabExam(hospitalization_id);**

Βοηθάει στη σύνδεση εργαστηριακών εξετάσεων με τη νοσηλεία στην οποία ανήκουν. Είναι χρήσιμο για την εμφάνιση του πλήρους ιστορικού μιας νοσηλείας.

- **CREATE INDEX idx_labexam_doctor ON LabExam(doctor_id);**

Χρησιμοποιείται για την αναζήτηση εργαστηριακών εξετάσεων που παρήγγειλε συγκεκριμένος γιατρός. Είναι χρήσιμο σε queries που συνδέουν γιατρούς με εξετάσεις ασθενών.

- **CREATE INDEX idx_docrat_doctor ON Doctor_Ratings(doctor_id);**

Το index αυτό βοηθάει στην αναζήτηση αξιολογήσεων ανά γιατρό. Είναι χρήσιμο στο Q4, όπου για συγκεκριμένο γιατρό υπολογίζεται ο μέσος όρος αξιολογήσεων των ασθενών του ως προς το κριτήριο ποιότητας ιατρικής φροντίδας.

- **CREATE INDEX idx_docrat_hosp ON Doctor_Ratings(hospitalization_id);**

Βοηθάει στη σύνδεση αξιολογήσεων γιατρών με συγκεκριμένες νοσηλείες. Είναι χρήσιμο όταν θέλουμε να βρούμε ποιες αξιολογήσεις γιατρών αντιστοιχούν σε μια νοσηλεία.

- **CREATE INDEX idx_hosprat_hosp**

ON Hospitalization_Ratings(hospitalization_id);

Επιτρέπει πιο γρήγορη σύνδεση της αξιολόγησης νοσηλείας με την αντίστοιχη νοσηλεία. Είναι χρήσιμο στο Q4 και στο Q6, όπου οι αξιολογήσεις συνδυάζονται με δεδομένα νοσηλειών και ασθενών.

Να σημειωθεί ότι τα indexes βελτιώνουν την ταχύτητα αναζήτησης, ωστόσο δεν πρέπει να χρησιμοποιούνται χωρίς λόγο, καθώς αυξάνουν τον αποθηκευτικό χώρο και κάνουν λίγο πιο αργές τις εισαγωγές και τις ενημερώσεις των δεδομένων.

9. Φόρτωση δεδομένων

Η φόρτωση των δεδομένων έγινε μέσω του αρχείου `load.sql` και με βοηθητικά Python scripts (`split_csvs.py`, `fake_data.py`, `download_images.py`). Τα δεδομένα χωρίζονται σε τρεις κατηγορίες: πραγματικά δεδομένα αναφοράς από αρχεία του Υπουργείου Υγείας, ενδεικτικά δεδομένα λειτουργίας και εικόνες για τις ανάγκες της web εφαρμογής. Πιο αναλυτικά:

- Στην πρώτη κατηγορία ανήκουν τα πραγματικά δεδομένα αναφοράς. Σε αυτά περιλαμβάνονται οι πίνακες `ICD10_Ref`, `KEN_Ref`, `LabExam_Ref`, `MedicalAct_Ref`, `Medications` και `Active_Substances`. Τα αρχικά αρχεία καθαρίστηκαν και επεξεργάστηκαν με το `split_csvs.py`, ώστε να κρατηθούν τα πεδία που χρειάζονται στη βάση και να είναι στη σωστή μορφή για εισαγωγή.

Πιο συγκεκριμένα, στη βάση φορτώθηκαν 11.008 διαγνώσεις ICD-10, 727 κωδικοί KEN, 3.036 εργαστηριακές εξετάσεις αναφοράς, 6.608 ιατρικές πράξεις αναφοράς, 161.787 φάρμακα και 12.077 δραστικές ουσίες. Για τα φάρμακα έγινε επίσης σύνδεση με τις δραστικές ουσίες τους μέσω του πίνακα `Medication_Substances`.

- Η δεύτερη κατηγορία αφορά τα ενδεικτικά δεδομένα λειτουργίας, τα οποία δημιουργήθηκαν με το `fake_data.py` και τη βιβλιοθήκη `Faker` της Python. Τα δεδομένα αυτά χρησιμοποιήθηκαν ώστε η βάση να έχει αρκετές εγγραφές για να μπορούν να ελεγχθούν οι συσχετίσεις, τα `triggers` και τα ζητούμενα `queries`.

Ειδικότερα, δημιουργήθηκαν 15 τμήματα, 10 αίθουσες επεμβάσεων, 150 μέλη προσωπικού, 200 ασθενείς, 100 κλίνες και 500 νοσηλείες. Από το προσωπικό, οι 80 είναι γιατροί, οι 50 νοσηλευτές και οι 20 διοικητικοί υπάλληλοι. Επιπλέον, δημιουργήθηκαν 500 `triage entries`, 1.350 βάρδιες, 300 συνταγογραφήσεις, 200 εργαστηριακές εξετάσεις, 150 ιατρικές πράξεις, 253 αξιολογήσεις νοσηλειών και 174 αξιολογήσεις γιατρών.

- Η τρίτη κατηγορία αφορά τις εικόνες που χρησιμοποιούνται στη web εφαρμογή. Οι εικόνες επιλέχθηκαν και αποθηκεύτηκαν με το `download_images.py` από το `Pexels` (δωρεάν πλατφόρμα φωτογραφιών με `copyright-free` περιεχόμενο) ώστε να αντιστοιχούν σε γιατρούς, νοσηλευτές, διοικητικούς υπαλλήλους, τμήματα και χώρους επεμβάσεων. Συνολικά, στον πίνακα `Images` καταχωρήθηκαν 174 εγγραφές.

Αξίζει να επισημάνουμε ότι η φόρτωση έγινε με συγκεκριμένη σειρά, ώστε να τηρούνται οι εξαρτήσεις μεταξύ των πινάκων. Πρώτα εισάγονται τα δεδομένα αναφοράς και οι βασικοί πίνακες, έπειτα οι πίνακες που εξαρτώνται από αυτούς και στο τέλος τα `junction tables`. Στο `load.sql` χρησιμοποιούνται εντολές `INSERT INTO`, ενώ στην αρχή ορίζεται `SET NAMES utf8mb4`, ώστε να υποστηρίζονται σωστά οι ελληνικοί χαρακτήρες.

10. Ερωτήματα (Queries)

Στην ενότητα αυτή παρουσιάζονται τα ερωτήματα που υλοποιήθηκαν για τη βάση δεδομένων του Γενικού Νοσοκομείου «Υγειόπολης». Συνδυάζουν δεδομένα από διαφορετικούς πίνακες και απαντούν στα ζητούμενα της εκφώνησης.

- **Query 1:**

Βρείτε τα συνολικά έσοδα του νοσοκομείου ανά τμήμα και ανά έτος, με ανάλυση ανά KEN κωδικό (βασικό κόστος έναντι πρόσθετης χρέωσης λόγω υπέρβασης ΜΔΝ) και κατανομή νοσηλειών ανά ασφαλιστικό φορέα.

```
SELECT
    d.name AS Department,
    YEAR(h.entry_date) AS Year,
    k.code AS KEN_Code,
    p.insurance_provider AS Provider,
    COUNT(h.id) AS Total_Hospitalizations,
    SUM(k.base_cost) AS Base_Revenue,
    SUM(CASE
        WHEN DATEDIFF(h.exit_date, h.entry_date) > k.mdn_days
        THEN (DATEDIFF(h.exit_date, h.entry_date) - k.mdn_days) *
(k.base_cost / k.mdn_days)
        ELSE 0
    END) AS Extra_Revenue,
    SUM(h.total_cost) AS Total_Revenue

FROM
    Hospitalizations h
JOIN
    Departments d ON h.department_id = d.id
JOIN
    KEN_Ref k ON h.ken_code = k.code
JOIN
    Patients p ON h.patient_id = p.id

GROUP BY
    d.name,
    YEAR(h.entry_date),
    k.code,
    p.insurance_provider

ORDER BY
    Year DESC,
    Department ASC,
    KEN_Code ASC;
```

Αρχικά κάνουμε JOIN από τον πίνακα Hospitalizations στους πίνακες Departments, KEN_Ref και Patients για να συνδυάσουμε τα στοιχεία των νοσηλειών με το όνομα του τμήματος, τα δεδομένα κόστους/ΜΔΝ του κωδικού KEN και τον ασφαλιστικό φορέα του κάθε ασθενούς. Ομαδοποιούμε τα αποτελέσματα με GROUP BY ανά τμήμα (d.name), ανά έτος

εισαγωγής (YEAR(h.entry_date)), ανά κωδικό KEN (k.code) και ανά ασφαλιστικό φορέα (p.insurance_provider).

Στο SELECT, υπολογίζουμε το πλήθος των νοσηλειών με την COUNT(h.id) και τα βασικά έσοδα αθροίζοντας το βασικό κόστος με την SUM(k.base_cost). Για τον υπολογισμό της πρόσθετης χρέωσης λόγω υπέρβασης των ημερών Μέσης Διάρκειας Νοσηλείας (ΜΔΝ), χρησιμοποιούμε μια δομή CASE WHEN. Ελέγχουμε αν η διαφορά των ημερών νοσηλείας (DATEDIFF(h.exit_date, h.entry_date)) είναι μεγαλύτερη από τις προβλεπόμενες ημέρες ΜΔΝ του κωδικού KEN (k.mdn_days). Αν ισχύει, αφαιρούμε τις ημέρες ΜΔΝ από τις συνολικές ημέρες νοσηλείας και πολλαπλασιάζουμε το αποτέλεσμα με το ημερήσιο κόστος ($k.base_cost / k.mdn_days$), αλλιώς επιστρέφουμε 0. Αθροίζουμε αυτά τα ποσά με SUM για να βρούμε τα πρόσθετα έσοδα (Extra_Revenue). Έπειτα, αθροίζουμε το συνολικό κόστος κάθε νοσηλείας με SUM(h.total_cost) για να εμφανίσουμε τα συνολικά έσοδα. Τέλος, με ORDER BY ταξινομούμε τα αποτελέσματα κατά φθίνουσα σειρά ως προς το έτος, και κατά αύξουσα σειρά ως προς το τμήμα και τον κωδικό KEN.

- **Query 2:**

Για συγκεκριμένη ειδικότητα ιατρού, βρείτε όλους τους ιατρούς που ανήκουν σε αυτήν, με ένδειξη αν είχαν εφημερία το τρέχον έτος και πόσες επεμβάσεις εκτέλεσαν ως κύριοι χειρουργοί.

```
SELECT
    s.last_name AS 'Επώνυμο',
    s.first_name AS 'Όνομα',
    d.specialty AS 'Ειδικότητα',
    CASE
        WHEN EXISTS (
            SELECT 1
            FROM Staff_Shifts ss
            JOIN Shifts sh ON ss.shift_id = sh.id
            WHERE ss.staff_id = d.staff_id
            AND YEAR(sh.shift_date) = YEAR(CURDATE())
        ) THEN 'ΝΑΙ'
        ELSE 'ΟΧΙ'
    END AS 'Εφημερία Τρέχον Έτος',
    (
        SELECT COUNT(*)
        FROM Medical_Acts ma
        WHERE ma.main_doctor_id = d.staff_id
    ) AS 'Πλήθος Επεμβάσεων (Κύριος Χειρουργός)'
FROM Doctors d
JOIN Staff s ON d.staff_id = s.id
WHERE d.specialty = 'Χειρουργική'
ORDER BY s.last_name, s.first_name;
```

Αρχικά κάνουμε JOIN από τον πίνακα Doctors στον πίνακα Staff για να αντλήσουμε τα προσωπικά στοιχεία των ιατρών. Φιλτράρουμε τα αποτελέσματα με WHERE d.specialty = 'Χειρουργική' ώστε να περιορίσουμε την αναζήτηση στη συγκεκριμένη ειδικότητα.

Στο SELECT, για να δείξουμε αν ο ιατρός είχε εφημερία το τρέχον έτος, χρησιμοποιούμε μια δομή CASE WHEN με EXISTS σε ένα συσχετισμένο υποερώτημα. Το υποερώτημα αυτό συνδέει τον πίνακα Staff_Shifts με τον πίνακα Shifts για τις βάρδιες του συγκεκριμένου ιατρού (ss.staff_id = d.staff_id) και ελέγχει αν το έτος της ημερομηνίας βάρδιας (YEAR(sh.shift_date)) ισούται με το τρέχον έτος (YEAR(CURDATE())). Αν βρεθεί τουλάχιστον μία τέτοια εγγραφή, εμφανίζεται η ένδειξη 'ΝΑΙ', διαφορετικά εμφανίζεται 'ΟΧΙ'. Επίσης, με ένα δεύτερο υποερώτημα, μετράμε το πλήθος των ιατρικών πράξεων/επεμβάσεων στον πίνακα Medical_Acts όπου ο συγκεκριμένος ιατρός συμμετείχε ως κύριος χειρουργός (ma.main_doctor_id = d.staff_id). Τέλος, με ORDER BY s.last_name, s.first_name εμφανίζουμε τα αποτελέσματα ταξινομημένα αλφαβητικά με βάση το επώνυμο και το όνομα του ιατρού.

- **Query 3:**

Βρείτε ποιοι ασθενείς έχουν νοσηλευτεί περισσότερες από 3 φορές στο ίδιο τμήμα, με το συνολικό κόστος νοσηλείας τους.

```
SELECT
    p.last_name AS Επώνυμο_Ασθενή,
    p.first_name AS Όνομα_Ασθενή,
    d.name AS Τμήμα,
    SUM(COALESCE(h.total_cost, 0)) AS Συνολικό_Κόστος
FROM Hospitalizations h
JOIN Patients p ON h.patient_id = p.id
JOIN Departments d ON h.department_id = d.id
GROUP BY p.id, d.id, p.last_name, p.first_name, d.name
HAVING COUNT(*) > 3
ORDER BY Συνολικό_Κόστος DESC;
```

Αρχικά κάνουμε JOIN από τον πίνακα Hospitalizations στους πίνακες Patients και Departments για να συνδυάσουμε τα στοιχεία των νοσηλειών με το ονοματεπώνυμο του ασθενούς και το όνομα του νοσοκομειακού τμήματος. Ομαδοποιούμε τις εγγραφές με GROUP BY ανά ασθενή και ανά τμήμα (p.id, d.id), συμπεριλαμβάνοντας στο group by και τα πεδία εμφάνισης (p.last_name, p.first_name, d.name).

Στο SELECT, υπολογίζουμε το συνολικό κόστος νοσηλείας χρησιμοποιώντας την SUM(COALESCE(h.total_cost, 0)), η οποία διασφαλίζει ότι αν το κόστος κάποιας νοσηλείας είναι NULL (δηλαδή δεν έχει καταχωρηθεί ακόμα), θα αντικατασταθεί με 0 πριν προστεθεί. Χρησιμοποιούμε την πρόταση HAVING COUNT(*) > 3 για να φιλτράρουμε τις ομάδες και να κρατήσουμε μόνο εκείνους τους συνδυασμούς ασθενούς - τμήματος που εμφανίζουν πάνω από 3 νοσηλείες. Τέλος, με ORDER BY Συνολικό_Κόστος DESC ταξινομούμε τα αποτελέσματα σε φθίνουσα σειρά ως προς το συνολικό κόστος, ώστε να εμφανίζονται πρώτοι οι ασθενείς με τα υψηλότερα έξοδα νοσηλείας.

❖ Σημείωση για τα Fake Data:

Επειδή η πιθανότητα να νοσηλευτεί τυχαία ένας ασθενής πάνω από 3 φορές στο ίδιο ακριβώς τμήμα είναι εξαιρετικά χαμηλή, έχει γίνει ειδική πρόβλεψη στο script fake_data.py. Συγκεκριμένα, εξαναγκάζουμε τις πρώτες 4 νοσηλείες της βάσης να αφορούν τον ασθενή με ID = 1 στο τμήμα με ID = 1 (Καρδιολογία), ώστε να διασφαλιστεί ότι το ερώτημα θα επιστρέψει αποτελέσματα κατά την εκτέλεσή του.

• Query 4:

Για συγκεκριμένο ιατρό, βρείτε τον μέσο όρο αξιολογήσεων των ασθενών του (κριτήριο: Ποιότητα ιατρικής φροντίδας) και τη Συνολική εντύπωση νοσηλείας.

Q4a - Απλή έκδοση

```
-- Τα αποτελέσματα των ANALYZE FORMAT=JSON και η σύγκριση
-- κόστους/χρόνου βρίσκονται αναλυτικά στο docs/report.pdf
-- ANALYZE FORMAT=JSON

SELECT
    s.last_name AS 'Επώνυμο Ιατρού',
    s.first_name AS 'Όνομα Ιατρού',
    AVG(dr.medical_care_quality) AS 'Μέση Ποιότητα Ιατρικής Φροντίδας',
    AVG(hr.overall_experience) AS 'Μέση Συνολική Εντύπωση Νοσηλείας'
FROM Doctor_Ratings dr
LEFT JOIN Hospitalization_Ratings hr ON dr.hospitalization_id =
hr.hospitalization_id
JOIN Staff s ON dr.doctor_id = s.id
WHERE dr.doctor_id = 2
GROUP BY dr.doctor_id, s.last_name, s.first_name;
```

Αρχικά, στην απλή έκδοση (Q4a), ξεκινάμε από τον πίνακα Doctor_Ratings και κάνουμε LEFT JOIN με τον πίνακα Hospitalization_Ratings χρησιμοποιώντας το πεδίο hospitalization_id (χρησιμοποιείται LEFT JOIN ώστε αν κάποια νοσηλεία δεν έχει γενική αξιολόγηση, να μη χαθεί η αξιολόγηση του ιατρού). Στη συνέχεια, κάνουμε JOIN με τον πίνακα Staff για να λάβουμε το ονοματεπώνυμο του ιατρού. Φιλτράρουμε τα δεδομένα για τον συγκεκριμένο ιατρό με την WHERE dr.doctor_id = 2 και ομαδοποιούμε με GROUP BY ανά ιατρό (dr.doctor_id, s.last_name, s.first_name). Στο SELECT, υπολογίζουμε τον μέσο όρο των αξιολογήσεων χρησιμοποιώντας τη συνάρτηση AVG για τα πεδία medical_care_quality (Ποιότητα Ιατρικής Φροντίδας) και overall_experience (Συνολική Εντύπωση Νοσηλείας).

Q4b - Εναλλακτική έκδοση με FORCE INDEX

```
-- Τα αποτελέσματα των ANALYZE FORMAT=JSON και η σύγκριση
-- κόστους/χρόνου βρίσκονται αναλυτικά στο docs/report.pdf
```

```
-- ANALYZE FORMAT=JSON
SELECT
    s.last_name AS 'Επώνυμο Ιατρού',
    s.first_name AS 'Όνομα Ιατρού',
    AVG(dr.medical_care_quality) AS 'Μέση Ποιότητα Ιατρικής Φροντίδας',
    AVG(hr.overall_experience) AS 'Μέση Συνολική Εντύπωση Νοσηλείας'
FROM Doctor_Ratings dr FORCE INDEX (idx_docrat_doctor)
LEFT JOIN Hospitalization_Ratings hr FORCE INDEX (idx_hosprat_hosp) ON
dr.hospitalization_id = hr.hospitalization_id
JOIN Staff s FORCE INDEX (PRIMARY) ON dr.doctor_id = s.id
WHERE dr.doctor_id = 2
GROUP BY dr.doctor_id, s.last_name, s.first_name;
```

Στην εναλλακτική έκδοση (Q4b), ακολουθούμε την ίδια ακριβώς λογική αλλά εισάγουμε την οδηγία `FORCE INDEX` για να εξαναγκάσουμε τον Query Optimizer να χρησιμοποιήσει συγκεκριμένα ευρετήρια (indexes), όπως το `idx_docrat_doctor` για το γρήγορο φιλτράρισμα του ιατρού, το `idx_hosprat_hosp` για τη συσχέτιση των πινάκων αξιολόγησης και το `PRIMARY` για τον πίνακα `Staff`.

Αποτελέσματα EXPLAIN ANALYZE

- Για το Q4a:

```
{
  "query_block": {
    "select_id": 1,
    "r_loops": 1,
    "r_total_time_ms": 0.0617,
    "table": {
      "table_name": "s",
      "access_type": "const",
      "possible_keys": ["PRIMARY"],
      "key": "PRIMARY",
      "key_length": "4",
      "used_key_parts": ["id"],
      "ref": ["const"],
      "r_loops": 0,
      "rows": 1,
      "r_rows": null,
      "filtered": 100,
      "r_filtered": null
    },
  },
  "table": {
    "table_name": "dr",
    "access_type": "ref",
  },
}
```



```

    "possible_keys": ["idx_docrat_doctor"],
    "key": "idx_docrat_doctor",
    "key_length": "4",
    "used_key_parts": ["doctor_id"],
    "ref": ["const"],
    "r_loops": 1,
    "rows": 1,
    "r_rows": 1,
    "r_total_time_ms": 0.0221,
    "filtered": 100,
    "r_filtered": 100
  },
  "table": {
    "table_name": "hr",
    "access_type": "eq_ref",
    "possible_keys": ["PRIMARY", "idx_hosprat_hosp"],
    "key": "PRIMARY",
    "key_length": "4",
    "used_key_parts": ["hospitalization_id"],
    "ref": ["hospitaldb.dr.hospitalization_id"],
    "r_loops": 1,
    "rows": 1,
    "r_rows": 0,
    "r_total_time_ms": 0.0047,
    "filtered": 100,
    "r_filtered": 100
  }
}

```

- Για το Q4b:

```

{
  "query_block": {
    "select_id": 1,
    "r_loops": 1,
    "r_total_time_ms": 0.6701,
    "table": {
      "table_name": "s",
      "access_type": "const",
      "possible_keys": ["PRIMARY"],
      "key": "PRIMARY",
      "key_length": "4",
      "used_key_parts": ["id"],
      "ref": ["const"],

```

```

    "r_loops": 0,
    "rows": 1,
    "r_rows": null,
    "filtered": 100,
    "r_filtered": null
  },
  "table": {
    "table_name": "dr",
    "access_type": "ref",
    "possible_keys": ["idx_docrat_doctor"],
    "key": "idx_docrat_doctor",
    "key_length": "4",
    "used_key_parts": ["doctor_id"],
    "ref": ["const"],
    "r_loops": 1,
    "rows": 1,
    "r_rows": 1,
    "r_total_time_ms": 0.0133,
    "filtered": 100,
    "r_filtered": 100
  },
  "table": {
    "table_name": "hr",
    "access_type": "ref",
    "possible_keys": ["idx_hosprat_hosp"],
    "key": "idx_hosprat_hosp",
    "key_length": "4",
    "used_key_parts": ["hospitalization_id"],
    "ref": ["hospitaldb.dr.hospitalization_id"],
    "r_loops": 1,
    "rows": 1,
    "r_rows": 0,
    "r_total_time_ms": 0.6317,
    "filtered": 100,
    "r_filtered": 100
  }
}
}

```

Σύγκριση εκτιμώμενου κόστους και πραγματικού χρόνου

Ο εκτιμώμενος αριθμός γραμμών που εξετάζεται είναι μικρός και στις δύο εκδόσεις, καθώς για τους βασικούς πίνακες του join εμφανίζεται rows = 1. Ωστόσο, οι πραγματικοί χρόνοι εκτέλεσης διαφέρουν σημαντικά. Στην απλή έκδοση Q4a ο συνολικός χρόνος εκτέλεσης είναι 0.0617 ms, ενώ στην έκδοση Q4b με FORCE INDEX ο συνολικός χρόνος εκτέλεσης

είναι 0.6701 ms. Επομένως, παρότι το εκτιμώμενο κόστος φαίνεται παρόμοιο ως προς τις γραμμές που εξετάζονται, η δεύτερη έκδοση είναι πιο αργή στην πράξη.

Ανάλυση / Συμπεράσματα

Στην πρώτη έκδοση (Q4a), ο optimizer της MariaDB επιλέγει αυτόματα πιο αποδοτικό τρόπο εκτέλεσης, χρησιμοποιώντας το Primary Key (PRIMARY) του πίνακα Hospitalization_Ratings για τη σύνδεση (access_type: eq_ref), με αποτέλεσμα ο χρόνος προσπέλασης του πίνακα να είναι μόλις 0.0047 ms.

Αντίθετα, στη δεύτερη έκδοση (Q4b), ο χειροκίνητος εξαναγκασμός χρήσης του δευτερεύοντος ευρετηρίου (FORCE INDEX (idx_hosprat_hosp)) υποβάθμισε την προσπέλαση σε απλό index lookup (access_type: ref). Η αλλαγή αυτή είχε ως συνέπεια ο χρόνος στο συγκεκριμένο βήμα να αυξηθεί στα 0.6317 ms, καθιστώντας το συγκεκριμένο βήμα του join πάνω από 130 φορές πιο αργό.

Συμπερασματικά, η αυτόματη επιλογή του RDBMS αποδείχθηκε ορθότερη, επιβεβαιώνοντας ότι η φυσική δομή των ευρετηρίων, δηλαδή τα Primary και Foreign Keys που σχεδιάστηκαν, λειτουργεί ήδη ικανοποιητικά χωρίς να απαιτούνται εξωτερικά hints.

- **Query 5:**

Βρείτε τους νέους ιατρούς (ηλικία < 35 ετών) που έχουν εκτελέσει τις περισσότερες πράξεις κεφαλαίου Β (Χειρουργικές/Επεμβατικές/Ενδοσκοπικές) ως κύριοι χειρουργοί.

```
SELECT
    s.last_name AS 'Επώνυμο',
    s.first_name AS 'Όνομα',
    TIMESTAMPDIFF(YEAR, s.birth_date, CURDATE()) AS 'Ηλικία',
    COUNT(ma.id) AS 'Πλήθος Χειρουργείων'
FROM Medical_Acts ma
JOIN MedicalAct_Ref mar ON ma.act_code = mar.code
JOIN Staff s ON ma.main_doctor_id = s.id
WHERE
    mar.category LIKE 'B.%'
    AND TIMESTAMPDIFF(YEAR, s.birth_date, CURDATE()) < 35
GROUP BY s.id, s.last_name, s.first_name, s.birth_date
ORDER BY COUNT(ma.id) DESC;
```

Αρχικά κάνουμε JOIN από τον πίνακα Medical_Acts στον πίνακα αναφοράς MedicalAct_Ref για να ελέγξουμε την κατηγορία της ιατρικής πράξης και στον πίνακα Staff για να αντλήσουμε τα στοιχεία των ιατρών που εκτέλεσαν την πράξη ως κύριοι χειρουργοί. Φιλτράρουμε τις εγγραφές στο WHERE θέτοντας δύο κριτήρια. Πρώτον η κατηγορία της ιατρικής πράξης να ξεκινάει από "B." (mar.category LIKE 'B.%'), η οποία αντιστοιχεί στις χειρουργικές επεμβάσεις σύμφωνα με την επίσημη ταξινόμηση του Υπουργείου Υγείας, και δεύτερον, ο ιατρός να είναι νέος σε ηλικία (κάτω των 35 ετών), υπολογίζοντας την ηλικία με τη συνάρτηση TIMESTAMPDIFF(YEAR, s.birth_date, CURDATE()) < 35.

Ομαδοποιούμε τις εγγραφές με GROUP BY ανά ιατρό (s.id) και τα προσωπικά του στοιχεία (s.last_name, s.first_name, s.birth_date). Στο SELECT, εμφανίζουμε το ονοματεπώνυμο, την ηλικία του ιατρού (με χρήση της TIMESTAMPDIFF) και το πλήθος των χειρουργείων χρησιμοποιώντας την COUNT(ma.id). Τέλος, με ORDER BY COUNT(ma.id) DESC ταξινομούμε τα αποτελέσματα σε φθίνουσα σειρά, ώστε οι ιατροί με τις περισσότερες επεμβάσεις να εμφανίζονται πρώτοι στην κορυφή της λίστας.

- **Query 6:**

Για συγκεκριμένο ασθενή, βρείτε το ιστορικό νοσηλειών του, τις αντίστοιχες διαγνώσεις (ICD-10), το συνολικό κόστος ανά νοσηλεία και τον μέσο όρο αξιολόγησής του.

Q6a - Απλή έκδοση

```
-- Τα αποτελέσματα των ANALYZE FORMAT=JSON και η σύγκριση
-- κόστους/χρόνου βρίσκονται αναλυτικά στο docs/report.pdf
-- ANALYZE FORMAT=JSON
```

SELECT

```
    h.id AS hospitalization_id,
    h.entry_date,
    h.exit_date,
    DATEDIFF(h.exit_date, h.entry_date) AS days_stayed,
    h.icd10_entry_code,
    icd_entry.description AS entry_diagnosis,
    h.icd10_exit_code,
    icd_exit.description AS exit_diagnosis,
    h.total_cost,
    ROUND(
        (hr.nursing_care_quality +
         hr.cleanliness +
         hr.food_quality +
         hr.overall_experience) / 4.0, 2) AS avg_hospitalization_rating,
    ROUND(AVG(dr.medical_care_quality), 2) AS avg_doctor_rating
FROM Hospitalizations h
LEFT JOIN ICD10_Ref icd_entry ON h.icd10_entry_code = icd_entry.code
LEFT JOIN ICD10_Ref icd_exit ON h.icd10_exit_code = icd_exit.code
LEFT JOIN Hospitalization_Ratings hr ON h.id = hr.hospitalization_id
LEFT JOIN Doctor_Ratings dr ON h.id = dr.hospitalization_id
WHERE h.patient_id = 1
GROUP BY
    h.id, h.entry_date, h.exit_date,
    h.icd10_entry_code, icd_entry.description,
    h.icd10_exit_code, icd_exit.description,
    h.total_cost,
    hr.nursing_care_quality, hr.cleanliness,
```

```
hr.food_quality, hr.overall_experience  
ORDER BY h.entry_date ASC;
```

Στην απλή έκδοση (Q6a), επιλέγουμε τα βασικά χαρακτηριστικά των νοσηλειών ενός συγκεκριμένου ασθενή, καθώς και τη διάρκεια κάθε νοσηλείας μέσω της συνάρτησης DATEDIFF. Επίσης, εμφανίζονται οι αντίστοιχες διαγνώσεις εισόδου και εξόδου από τον πίνακα ICD10_Ref, το συνολικό κόστος της νοσηλείας όπως έχει διαμορφωθεί με βάση το KEN, καθώς και οι αξιολογήσεις που σχετίζονται με τη νοσηλεία.

Για την άντληση αυτών των πληροφοριών ξεκινάμε από τον πίνακα Hospitalizations και κάνουμε LEFT JOIN με τον πίνακα ICD10_Ref δύο φορές, μία για τη διάγνωση εισόδου και μία για τη διάγνωση εξόδου. Επιπλέον, γίνεται LEFT JOIN με τους πίνακες Hospitalization_Ratings και Doctor_Ratings, ώστε να ληφθούν υπόψη τόσο οι γενικές αξιολογήσεις της νοσηλείας όσο και οι αξιολογήσεις των ιατρών. Χρησιμοποιούμε LEFT JOIN, επειδή θέλουμε να εμφανιστούν όλες οι νοσηλείες του ασθενή με ID = 1, ακόμα και αν για κάποιες από αυτές δεν υπάρχουν καταχωρημένες αξιολογήσεις ή πλήρη στοιχεία διαγνώσεων.

Q6b - Εναλλακτική έκδοση με FORCE INDEX

```
-- Τα αποτελέσματα των ANALYZE FORMAT=JSON και η σύγκριση  
-- κόστους/χρόνου βρίσκονται αναλυτικά στο docs/report.pdf  
-- ANALYZE FORMAT=JSON
```

```
SELECT  
  h.id AS hospitalization_id,  
  h.entry_date,  
  h.exit_date,  
  DATEDIFF(h.exit_date, h.entry_date) AS days_stayed,  
  h.icd10_entry_code,  
  icd_entry.description AS entry_diagnosis,  
  h.icd10_exit_code,  
  icd_exit.description AS exit_diagnosis,  
  h.total_cost,  
  ROUND(  
    (hr.nursing_care_quality +  
     hr.cleanliness +  
     hr.food_quality +  
     hr.overall_experience) / 4.0, 2) AS avg_hospitalization_rating,  
  ROUND(AVG(dr.medical_care_quality), 2) AS avg_doctor_rating  
FROM Hospitalizations h FORCE INDEX (idx_hosp_patient)  
LEFT JOIN ICD10_Ref icd_entry ON h.icd10_entry_code = icd_entry.code  
LEFT JOIN ICD10_Ref icd_exit ON h.icd10_exit_code = icd_exit.code  
LEFT JOIN Hospitalization_Ratings hr FORCE INDEX (idx_hosprat_hosp) ON  
h.id = hr.hospitalization_id
```

```

LEFT JOIN Doctor_Ratings dr FORCE INDEX (idx_docrat_hosp) ON h.id =
dr.hospitalization_id
WHERE h.patient_id = 1
GROUP BY
    h.id, h.entry_date, h.exit_date,
    h.icd10_entry_code, icd_entry.description,
    h.icd10_exit_code, icd_exit.description,
    h.total_cost,
    hr.nursing_care_quality, hr.cleanliness,
    hr.food_quality, hr.overall_experience
ORDER BY h.entry_date ASC;

```

Στην εναλλακτική έκδοση (Q6b), ακολουθούμε την ίδια λογική με την απλή έκδοση, αλλά εισάγουμε την οδηγία **FORCE INDEX** για να εξαναγκάσουμε τον Query Optimizer να χρησιμοποιήσει συγκεκριμένα indexes.

Αποτελέσματα EXPLAIN ANALYZE

- Για το Q6a:

```

{
  "query_block": {
    "select_id": 1,
    "r_loops": 1,
    "r_total_time_ms": 8.463,
    "filesort": {
      "sort_key": "h.entry_date",
      "r_loops": 1,
      "r_total_time_ms": 0.023,
      "r_used_priority_queue": false,
      "r_output_rows": 6,
      "r_buffer_size": "320",
      "temporary_table": {
        "table": {
          "table_name": "h",
          "access_type": "ref",
          "possible_keys": ["idx_hosp_patient"],
          "key": "idx_hosp_patient",
          "key_length": "4",
          "used_key_parts": ["patient_id"],
          "ref": ["const"],
          "r_loops": 1,
          "rows": 6,
          "r_rows": 6,
          "r_total_time_ms": 0.0551,
          "filtered": 100,

```

```

    "r_filtered": 100,
    "attached_condition": "h.patient_id <=> 1"
  },
  "table": {
    "table_name": "icd_entry",
    "access_type": "ref",
    "possible_keys": ["code"],
    "key": "code",
    "key_length": "402",
    "used_key_parts": ["code"],
    "ref": ["hospitaldb.h.icd10_entry_code"],
    "r_loops": 6,
    "rows": 1,
    "r_rows": 1,
    "r_total_time_ms": 0.0785,
    "filtered": 100,
    "r_filtered": 100,
    "attached_condition": "trigcond(trigcond(h.icd10_entry_code is
not null))"
  },
  "table": {
    "table_name": "icd_exit",
    "access_type": "ref",
    "possible_keys": ["code"],
    "key": "code",
    "key_length": "402",
    "used_key_parts": ["code"],
    "ref": ["hospitaldb.h.icd10_exit_code"],
    "r_loops": 6,
    "rows": 1,
    "r_rows": 1,
    "r_total_time_ms": 0.0691,
    "filtered": 100,
    "r_filtered": 100,
    "attached_condition": "trigcond(trigcond(h.icd10_exit_code is
not null))"
  },
  "table": {
    "table_name": "hr",
    "access_type": "eq_ref",
    "possible_keys": ["PRIMARY", "idx_hosprat_hosp"],
    "key": "PRIMARY",
    "key_length": "4",
    "used_key_parts": ["hospitalization_id"],
    "ref": ["hospitaldb.h.id"],
    "r_loops": 6,

```

```

    "rows": 1,
    "r_rows": 0.1667,
    "r_total_time_ms": 0.0202,
    "filtered": 100,
    "r_filtered": 100
  },
  "table": {
    "table_name": "dr",
    "access_type": "ref",
    "possible_keys": ["PRIMARY", "idx_docrat_hosp"],
    "key": "PRIMARY",
    "key_length": "4",
    "used_key_parts": ["hospitalization_id"],
    "ref": ["hospitaldb.h.id"],
    "r_loops": 6,
    "rows": 1,
    "r_rows": 1.3333,
    "r_total_time_ms": 0.039,
    "filtered": 100,
    "r_filtered": 100
  }
}
}
}
}
}

```

- Για το Q6b:

```

{
  "query_block": {
    "select_id": 1,
    "r_loops": 1,
    "r_total_time_ms": 12.048,
    "filesort": {
      "sort_key": "h.entry_date",
      "r_loops": 1,
      "r_total_time_ms": 0.0411,
      "r_used_priority_queue": false,
      "r_output_rows": 6,
      "r_buffer_size": "320",
      "temporary_table": {
        "table": {
          "table_name": "h",
          "access_type": "ref",
          "possible_keys": ["idx_hosp_patient"],

```



```

    "key": "idx_hosp_patient",
    "key_length": "4",
    "used_key_parts": ["patient_id"],
    "ref": ["const"],
    "r_loops": 1,
    "rows": 6,
    "r_rows": 6,
    "r_total_time_ms": 0.9974,
    "filtered": 100,
    "r_filtered": 100,
    "attached_condition": "h.patient_id <=> 1"
  },
  "table": {
    "table_name": "icd_entry",
    "access_type": "ref",
    "possible_keys": ["code"],
    "key": "code",
    "key_length": "402",
    "used_key_parts": ["code"],
    "ref": ["hospitaldb.h.icd10_entry_code"],
    "r_loops": 6,
    "rows": 1,
    "r_rows": 1,
    "r_total_time_ms": 2.0888,
    "filtered": 100,
    "r_filtered": 100,
    "attached_condition": "trigcond(trigcond(h.icd10_entry_code is
not null))"
  },
  "table": {
    "table_name": "icd_exit",
    "access_type": "ref",
    "possible_keys": ["code"],
    "key": "code",
    "key_length": "402",
    "used_key_parts": ["code"],
    "ref": ["hospitaldb.h.icd10_exit_code"],
    "r_loops": 6,
    "rows": 1,
    "r_rows": 1,
    "r_total_time_ms": 2.8132,
    "filtered": 100,
    "r_filtered": 100,
    "attached_condition": "trigcond(trigcond(h.icd10_exit_code is
not null))"
  },

```

```

"table": {
  "table_name": "hr",
  "access_type": "ref",
  "possible_keys": ["idx_hosprat_hosp"],
  "key": "idx_hosprat_hosp",
  "key_length": "4",
  "used_key_parts": ["hospitalization_id"],
  "ref": ["hospitaldb.h.id"],
  "r_loops": 6,
  "rows": 1,
  "r_rows": 0.1667,
  "r_total_time_ms": 0.036,
  "filtered": 100,
  "r_filtered": 100
},
"table": {
  "table_name": "dr",
  "access_type": "ref",
  "possible_keys": ["idx_docrat_hosp"],
  "key": "idx_docrat_hosp",
  "key_length": "4",
  "used_key_parts": ["hospitalization_id"],
  "ref": ["hospitaldb.h.id"],
  "r_loops": 6,
  "rows": 1,
  "r_rows": 1.3333,
  "r_total_time_ms": 0.3375,
  "filtered": 100,
  "r_filtered": 100
}
}
}
}
}
}

```

Σύγκριση εκτιμώμενου κόστους και πραγματικού χρόνου

Σύμφωνα με τα παραπάνω αποτελέσματα, εξετάζονται 6 νοσηλείες του συγκεκριμένου ασθενή και στις δύο εκδόσεις, καθώς στον πίνακα Hospitalizations εμφανίζεται rows = 6 και r_rows = 6. Το query χρησιμοποιεί προσωρινό πίνακα και filesort για την ταξινόμηση των νοσηλειών με βάση την ημερομηνία εισαγωγής. Στην απλή έκδοση Q6a ο συνολικός χρόνος εκτέλεσης είναι 8.463 ms, ενώ στην εναλλακτική έκδοση Q6b με FORCE INDEX ο συνολικός χρόνος εκτέλεσης είναι 12.048 ms. Επομένως, η δεύτερη έκδοση είναι πιο αργή στην πράξη, παρότι εξετάζει παρόμοιο πλήθος γραμμών.

Ανάλυση / Συμπεράσματα

Στο συγκεκριμένο ερώτημα, ο optimizer της MariaDB παράγει έναν προσωρινό πίνακα και εκτελεί filesort για να ταξινομήσει τις 6 νοσηλείες του ασθενούς βάσει ημερομηνίας εισαγωγής.

Στην πρώτη έκδοση (Q6a), η σύνδεση με τον πίνακα Hospitalization_Ratings γίνεται με πιο αποδοτικό τρόπο, μέσω του Primary Key, με access_type = eq_ref. Αντίθετα, στη δεύτερη έκδοση (Q6b), η χρήση FORCE INDEX υποβαθμίζει αυτή την προσπάθεια σε ref μέσω του δευτερεύοντος index idx_hosprat_hosp.

Η χειροκίνητη επιβολή των indexes στη δεύτερη έκδοση επηρέασε αρνητικά και τον πίνακα Doctor_Ratings, αυξάνοντας τον χρόνο αναζήτησης από 0.0390 ms σε 0.3375 ms. Συνολικά, ο χρόνος εκτέλεσης αυξήθηκε από 8.463 ms σε 12.048 ms, δηλαδή περίπου κατά 42%. Επομένως, η αυτόματη επιλογή της MariaDB αποδείχθηκε καλύτερη από τη χειροκίνητη χρήση FORCE INDEX (το ίδιο προέκυψε και στο query 4).

- **Query 7:**

Βρείτε για κάθε δραστική ουσία τον αριθμό ασθενών που έχουν δηλώσει αλλεργία και τον αριθμό φαρμάκων που την περιέχουν, ταξινομημένα κατά συνολικό αριθμό αλλεργικών ασθενών.

```
SELECT
    s.id AS substance_id,
    substance_name,
    COUNT(DISTINCT p.patient_id) AS num_allergic_patients,
    COUNT(DISTINCT m.medication_id) AS num_medications_containing
FROM Active_Substances s
LEFT JOIN Patient_Allergies p ON s.id = p.substance_id
LEFT JOIN Medication_Substances m ON s.id = m.substance_id
GROUP BY
    s.id, s.substance_name
ORDER BY num_allergic_patients DESC;
```

Αρχικά κάνουμε LEFT JOIN από τον πίνακα Active_Substances στους πίνακες Patient_Allergies και Medication_Substances, ώστε να εξετάσουμε κάθε δραστική ουσία, ανεξάρτητα από το αν υπάρχουν ασθενείς αλλεργικοί σε αυτήν ή αν περιέχεται σε κάποιο φάρμακο. Κρατάμε το id και το όνομα της δραστικής ουσίας και μετράμε πόσοι ασθενείς έχουν δηλώσει αλλεργία σε αυτήν, καθώς και σε πόσα φάρμακα περιέχεται.

Χρησιμοποιούμε COUNT(DISTINCT ...), ώστε να μη γίνονται διπλές μετρήσεις λόγω των πολλαπλών LEFT JOIN. Τέλος, με ORDER BY ταξινομούμε τα αποτελέσματα σε φθίνουσα σειρά με βάση τον αριθμό των αλλεργικών ασθενών.

- **Query 8:**

Βρείτε το προσωπικό (ιατροί, νοσηλευτές, διοικητικό προσωπικό) που δεν έχει προγραμματισμένη εφημερία σε συγκεκριμένη ημερομηνία και τμήμα.

```

SELECT
    s.id AS staff_id,
    s.amka,
    s.first_name,
    s.last_name,
    s.staff_type,
    s.birth_date,
    TIMESTAMPDIFF(YEAR, s.birth_date, CURDATE()) AS age,
    s.email,
    s.phone,
    s.hire_date,
    d.license_number,
    d.specialty,
    d.rank AS doctor_rank,
    d.supervisor_id,
    n.rank AS nurse_rank,
    a.duty_role,
    a.office_location,
    CASE s.staff_type
        WHEN 'doctor' THEN
            (SELECT GROUP_CONCAT(dep.name SEPARATOR ', ')
             FROM Doctor_Departments dd
             JOIN Departments dep ON dd.department_id = dep.id
             WHERE dd.doctor_id = s.id)
        WHEN 'nurse' THEN
            (SELECT dep.name FROM Departments dep
             WHERE dep.id = n.department_id)
        WHEN 'admin' THEN
            (SELECT dep.name FROM Departments dep
             WHERE dep.id = a.department_id)
        END AS belongs_to_departments
FROM Staff s
LEFT JOIN Doctors d ON s.id = d.staff_id
LEFT JOIN Nurses n ON s.id = n.staff_id
LEFT JOIN Administrative_Staff a ON s.id = a.staff_id
JOIN Departments dep ON dep.name = 'Ακτινολογικό'
WHERE s.active = TRUE
    AND (
        EXISTS (SELECT 1 FROM Doctor_Departments dd
                 WHERE dd.doctor_id = s.id AND dd.department_id =
dep.id)
        OR (n.department_id = dep.id)
        OR (a.department_id = dep.id)
    )
    AND s.id NOT IN (

```

```

SELECT ss.staff_id
FROM Staff_Shifts ss
JOIN Shifts sh ON ss.shift_id = sh.id
WHERE sh.shift_date = '2026-05-01' AND sh.department_id = dep.id
)
ORDER BY s.staff_type, s.last_name, s.first_name;

```

Αρχικά ξεκινάμε από τον πίνακα Staff και κάνουμε LEFT JOIN με τους πίνακες Doctors, Nurses και Administrative_Staff, ώστε να πάρουμε τα στοιχεία κάθε μέλους προσωπικού ανάλογα με την κατηγορία του. Στη συνέχεια, συνδέουμε το query με το τμήμα Ακτινολογικό, ώστε να εξετάσουμε μόνο το προσωπικό που ανήκει στο συγκεκριμένο τμήμα και είναι ενεργό.

Το query ελέγχει ότι το συγκεκριμένο μέλος προσωπικού δεν εμφανίζεται στον πίνακα Staff_Shifts για βάρδια του ίδιου τμήματος και της ημερομηνίας 2026-05-01. Έτσι επιστρέφονται μόνο τα μέλη προσωπικού που δεν έχουν προγραμματισμένη εφημερία εκείνη την ημέρα στο συγκεκριμένο τμήμα. Για τους γιατρούς που ανήκουν σε περισσότερα από ένα τμήματα, χρησιμοποιείται GROUP_CONCAT, ώστε τα τμήματα να εμφανίζονται συγκεντρωμένα στην ίδια γραμμή.

- **Query 9:**

Βρείτε ποιοι ασθενείς νοσηλεύτηκαν τον ίδιο αριθμό ημερών σε διάστημα ενός έτους, με συνολική διάρκεια άνω των 15 ημερών.

```

WITH patient_yearly_hospitalized AS (
    SELECT
        h.patient_id,
        YEAR(h.entry_date) AS stay_year,
        SUM(DATEDIFF(COALESCE(h.exit_date, CURDATE()), h.entry_date)) AS
total_days
    FROM Hospitalizations h
    GROUP BY
        h.patient_id,
        YEAR(h.entry_date)
    HAVING total_days > 15
)
SELECT
    p1.patient_id AS patient1_id,
    s1.first_name AS patient1_first_name,
    s1.last_name AS patient1_last_name,
    p2.patient_id AS patient2_id,
    s2.first_name AS patient2_first_name,
    s2.last_name AS patient2_last_name,
    p1.total_days,
    p1.stay_year

```

```

FROM patient_yearly_hospitalized p1 JOIN patient_yearly_hospitalized p2
ON p1.total_days = p2.total_days
    AND p1.stay_year = p2.stay_year
    AND p1.patient_id < p2.patient_id
JOIN Patients s1 ON p1.patient_id = s1.id
JOIN Patients s2 ON p2.patient_id = s2.id
ORDER BY p1.stay_year, p1.total_days;

```

Σε αυτό το query, χρησιμοποιούμε WITH ώστε να φτιάξουμε ένα Common Table Expression (CTE) με το όνομα patient_yearly_hospitalized. Εκεί κρατάμε για κάθε ασθενή το έτος, τα στοιχεία του και τις συνολικές ημέρες νοσηλείας του μέσα σε εκείνο το έτος, τις οποίες υπολογίζουμε με DATEDIFF. Με HAVING κρατάμε μόνο όσους έχουν συνολική διάρκεια νοσηλείας πάνω από 15 ημέρες.

Στη συνέχεια κάνουμε JOIN τον πίνακα αυτόν με τον εαυτό του, για να βρούμε ασθενείς που μέσα στο ίδιο έτος είχαν τον ίδιο συνολικό αριθμό ημερών νοσηλείας. Η συνθήκη p1.patient_id < p2.patient_id μπαίνει για να μη συγκρίνεται ένας ασθενής με τον εαυτό του και για να μην εμφανίζονται διπλές εγγραφές (δηλαδή πρώτα ο A με τον B και μετά ο B με τον A). Έτσι, κάθε ζευγάρι ασθενών εμφανίζεται μόνο μία φορά, μαζί με τα στοιχεία και των δύο.

- **Query 10:**

Πολλοί ασθενείς λαμβάνουν συνδυασμούς φαρμάκων. Βρείτε τα top-3 ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα στον ίδιο ασθενή κατά την ίδια νοσηλεία, ταξινομημένα κατά συχνότητα εμφάνισης.

```

WITH prescribed_substances AS (
    SELECT
        p.hospitalization_id,
        p.patient_id,
        ms.substance_id,
        s.substance_name
    FROM Prescriptions p
    JOIN Medication_Substances ms ON p.medication_id = ms.medication_id
    JOIN Active_Substances s ON ms.substance_id = s.id
)
SELECT
    ps1.substance_name AS substance_1,
    ps2.substance_name AS substance_2,
    COUNT(*) AS frequency
FROM prescribed_substances ps1 JOIN prescribed_substances ps2
ON ps1.hospitalization_id = ps2.hospitalization_id
AND ps1.patient_id = ps2.patient_id
AND ps1.substance_id < ps2.substance_id
GROUP BY
    ps1.substance_id, ps1.substance_name, ps2.substance_id,
    ps2.substance_name

```

```
ORDER BY frequency DESC
LIMIT 3;
```

Χρησιμοποιούμε WITH ώστε να φτιάξουμε ένα CTE με τις δραστικές ουσίες που έχουν συνταγογραφηθεί. Σε αυτόν κρατάμε τον ασθενή, τη νοσηλεία και τη δραστική ουσία κάθε φαρμάκου, ώστε στη συνέχεια να μπορούμε να ελέγξουμε ποιοι συνδυασμοί εμφανίζονται μαζί.

Έπειτα κάνουμε JOIN τον ενδιαμέσου πίνακα με τον εαυτό του, ώστε να βρούμε ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν στον ίδιο ασθενή και στην ίδια νοσηλεία. Η συνθήκη με < χρησιμοποιείται για να μη συγκρίνεται μια δραστική ουσία με τον εαυτό της και για να μην εμφανίζονται διπλές εγγραφές (δηλαδή και «ουσία A - ουσία B» και «ουσία B - ουσία A»). Τέλος, με ORDER BY ταξινομούμε τα ζεύγη κατά συχνότητα εμφάνισης και με LIMIT 3 κρατάμε τα τρία πιο συχνά.

❖ Σημείωση για τα Fake Data:

Ισχύει ότι η πιθανότητα να προκύψουν τυχαία επαναλαμβανόμενα ζεύγη δραστικών ουσιών που έχουν συνταγογραφηθεί στον ίδιο ασθενή κατά την ίδια νοσηλεία είναι σχετικά χαμηλή, επομένως τροποποιήσαμε το script fake_data.py προκειμένου να δημιουργεί σκόπιμα ορισμένες τέτοιες περιπτώσεις, για τις ανάγκες του ερωτήματος. Συγκεκριμένα, στις πρώτες συνταγογραφήσεις της βάσης χρησιμοποιούνται συγκεκριμένα ζεύγη φαρμάκων στις ίδιες νοσηλείες, ώστε μέσω των δραστικών ουσιών τους να εμφανίζονται σταθεροί συνδυασμοί. Με αυτόν τον τρόπο διασφαλίζεται ότι το ερώτημα επιστρέφει αποτελέσματα και μπορούν να προκύψουν τα top 3 ζεύγη δραστικών ουσιών.

• Query 11:

Βρείτε όλους τους ιατρούς που έχουν εκτελέσει τουλάχιστον 5 λιγότερες ιατρικές πράξεις από τον ιατρό με τις περισσότερες στο τρέχον έτος.

```
SELECT
    d.staff_id AS 'ID Ιατρού',
    s.last_name AS 'Επώνυμο',
    s.first_name AS 'Όνομα',
    d.specialty AS 'Ειδικότητα',
    d.rank AS 'Βαθμίδα',
    COUNT(ma.id) AS 'Ιατρικές πράξεις τρέχον έτος'
FROM Doctors d
JOIN Staff s ON d.staff_id = s.id
LEFT JOIN Medical_Acts ma
    ON ma.main_doctor_id = d.staff_id
    AND YEAR(ma.scheduled_time) = YEAR(CURDATE())
GROUP BY d.staff_id, s.last_name, s.first_name, d.specialty, d.rank
HAVING COUNT(ma.id) <= (
    SELECT COUNT(id)
    FROM Medical_Acts
    WHERE YEAR(scheduled_time) = YEAR(CURDATE())
```

```

GROUP BY main_doctor_id
ORDER BY COUNT(id) DESC
LIMIT 1
) -5
ORDER BY COUNT(ma.id) DESC, s.last_name;

```

Αρχικά κάνουμε JOIN από τον πίνακα Doctors στον πίνακα Staff, ώστε να πάρουμε τα βασικά στοιχεία κάθε ιατρού (ονοματεπώνυμο, ειδικότητα και βαθμίδα). Στη συνέχεια κάνουμε LEFT JOIN με τον πίνακα Medical_Acts, κρατώντας μόνο τις ιατρικές πράξεις του τρέχοντος έτους με τη συνθήκη YEAR(ma.scheduled_time) = YEAR(CURDATE()). Χρησιμοποιείται LEFT JOIN, ώστε να εμφανίζονται και οι ιατροί που δεν έχουν εκτελέσει καμία πράξη μέσα στο τρέχον έτος.

Με GROUP BY ομαδοποιούμε τα αποτελέσματα ανά γιατρό και με COUNT(ma.id) υπολογίζουμε πόσες ιατρικές πράξεις έχει εκτελέσει ο καθένας ως κύριος χειρουργός. Στο HAVING συγκρίνουμε αυτό το πλήθος με το μέγιστο πλήθος επεμβάσεων που έχει εκτελέσει κάποιος γιατρός μέσα στο τρέχον έτος, το οποίο υπολογίζεται με subquery. Κρατάμε μόνο τους γιατρούς που έχουν τουλάχιστον 5 λιγότερες πράξεις από τον ιατρό με τις περισσότερες πράξεις. Τέλος, με ORDER BY ταξινομούμε τα αποτελέσματα με βάση το πλήθος των πράξεων και έπειτα αλφαβητικά με βάση το επώνυμο.

❖ Σημείωση για τα Fake Data:

Το Query 11 βασίζεται στη σύγκριση του αριθμού επεμβάσεων ανά ιατρό. Για τον λόγο αυτό, τα fake data έχουν διαμορφωθεί έτσι ώστε οι ιατρικές πράξεις να μην κατανέμονται ισόποσα σε όλους τους ιατρούς. Σε ορισμένες περιπτώσεις χρησιμοποιούνται συχνότερα κάποιοι από τους πρώτους ιατρούς της βάσης, ώστε να υπάρχουν μεγαλύτερες διαφορές στον αριθμό επεμβάσεων. Έτσι το ερώτημα επιστρέφει αρκετά αποτελέσματα και μπορεί να γίνει η σύγκριση με τους ιατρούς που έχουν τουλάχιστον 5 λιγότερες επεμβάσεις από τον πρώτο.

• Query 12:

Βρείτε τον απαιτούμενο αριθμό προσωπικού ανά τμήμα και ανά βάρδια για συγκεκριμένη εβδομάδα, με ανάλυση ανά υποκλάση προσωπικού (ιατροί ανά ειδικότητα, νοσηλευτές ανά βαθμίδα, διοικητικό προσωπικό ανά ρόλο).

```

-- Ιατροί ανά ειδικότητα
SELECT
    dep.name AS 'Τμήμα',
    sh.shift_date AS 'Ημερομηνία',
    sh.shift_type AS 'Βάρδια',
    'Ιατρός' AS 'Κατηγορία',
    d.specialty AS 'Υποκατηγορία',
    COUNT(ss.staff_id) AS 'Αριθμός Προσωπικού'
FROM Shifts sh
JOIN Departments dep ON sh.department_id = dep.id
JOIN Staff_Shifts ss ON ss.shift_id = sh.id

```



```

JOIN Doctors d ON d.staff_id = ss.staff_id
WHERE sh.shift_date BETWEEN '2026-05-04' AND '2026-05-10'
GROUP BY dep.name, sh.shift_date, sh.shift_type, d.specialty

```

```

UNION ALL

```

```

-- Νοσηλευτές ανά βαθμίδα

```

```

SELECT
    dep.name,
    sh.shift_date,
    sh.shift_type,
    'Νοσηλευτής',
    n.rank,
    COUNT(ss.staff_id)
FROM Shifts sh
JOIN Departments dep ON sh.department_id = dep.id
JOIN Staff_Shifts ss ON ss.shift_id = sh.id
JOIN Nurses n ON n.staff_id = ss.staff_id
WHERE sh.shift_date BETWEEN '2026-05-04' AND '2026-05-10'
GROUP BY dep.name, sh.shift_date, sh.shift_type, n.rank

```

```

UNION ALL

```

```

-- Διοικητικό προσωπικό ανά ρόλο

```

```

SELECT
    dep.name,
    sh.shift_date,
    sh.shift_type,
    'Διοικητικό προσωπικό',
    a.duty_role,
    COUNT(ss.staff_id)
FROM Shifts sh
JOIN Departments dep ON sh.department_id = dep.id
JOIN Staff_Shifts ss ON ss.shift_id = sh.id
JOIN Administrative_Staff a ON a.staff_id = ss.staff_id
WHERE sh.shift_date BETWEEN '2026-05-04' AND '2026-05-10'
GROUP BY dep.name, sh.shift_date, sh.shift_type, a.duty_role

```

```

ORDER BY Ημερομηνία, Τμήμα, Βάρδια, Κατηγορία, Υποκατηγορία;

```

Το Query 12 εμφανίζει πόσα μέλη προσωπικού είναι προγραμματισμένα σε κάθε τμήμα και βάρδια για το διάστημα από 2026-05-04 έως και 2026-05-10. Ξεκινάμε από τον πίνακα Shifts και τον συνδέουμε με τους πίνακες Departments και Staff_Shifts, ώστε να γνωρίζουμε σε ποιο τμήμα και σε ποια βάρδια αντιστοιχεί κάθε μέλος προσωπικού.

Το ερώτημα χωρίζεται σε τρία μέρη, τα οποία ενώνονται με UNION ALL. Στο πρώτο μέρος μετράμε τους ιατρούς ανά ειδικότητα, στο δεύτερο τους νοσηλευτές ανά βαθμίδα και στο τρίτο το διοικητικό προσωπικό ανά ρόλο. Σε καθεμία από τις παραπάνω περιπτώσεις γίνεται GROUP BY με βάση το τμήμα, την ημερομηνία, τον τύπο βάρδιας και την αντίστοιχη υποκατηγορία προσωπικού.

Τέλος, με ORDER BY ταξινομούμε τα αποτελέσματα ανά ημερομηνία, τμήμα, βάρδια, κατηγορία και υποκατηγορία, ώστε η κατανομή του προσωπικού να παρουσιάζεται πιο ευανάγνωστα.

❖ Σημείωση:

Στο Query 12 ο «απαιτούμενος αριθμός προσωπικού» ερμηνεύεται ως το προσωπικό που απαιτείται στην πράξη για την κάλυψη των βάρδιών, δηλαδή το προσωπικό που έχει προγραμματιστεί σε κάθε βάρδια. Η επιλογή αυτή έγινε επειδή το ερώτημα αφορά συγκεκριμένο χρονικό διάστημα και ζητά ανάλυση ανά τμήμα, ημερομηνία, βάρδια και κατηγορία προσωπικού. Αν θέλαμε μόνο τα ελάχιστα απαιτούμενα άτομα ανά τμήμα, θα αρκούσαν τα αντίστοιχα πεδία του πίνακα Departments. Ωστόσο, εδώ επιλέγεται να εξεταστεί το πρόγραμμα βάρδιών, ώστε να φανεί πόσα άτομα έχουν πράγματι ανατεθεί ανά ημερομηνία, βάρδια, τμήμα και κατηγορία προσωπικού.

• **Query 13:**

Βρείτε για κάθε ιατρό όλη την ιεραρχία εποπτείας του, από τον άμεσο επόπτη έως τον Διευθυντή, με ένδειξη του επιπέδου σε κάθε βαθμίδα.

```
WITH RECURSIVE supervision_hierarchy (level, doctor_id, supervisor_id) AS
(
    SELECT 1, d.staff_id, d.supervisor_id
    FROM Doctors d
    WHERE d.supervisor_id IS NOT NULL
    UNION ALL
    SELECT sh.level + 1, sh.doctor_id, d.supervisor_id
    FROM Doctors d
    INNER JOIN supervision_hierarchy sh ON d.staff_id = sh.supervisor_id
    WHERE d.supervisor_id IS NOT NULL
)
SELECT
    s_doc.last_name AS 'Επώνυμο Ιατρού',
    s_doc.first_name AS 'Όνομα Ιατρού',
    d_doc.rank AS 'Βαθμίδα Ιατρού',
    level AS 'Επίπεδο Εποπτείας',
    s_sup.last_name AS 'Επώνυμο Επόπτη',
    s_sup.first_name AS 'Όνομα Επόπτη',
    d_sup.rank AS 'Βαθμίδα Επόπτη'
FROM supervision_hierarchy sh
JOIN Doctors d_doc ON d_doc.staff_id = sh.doctor_id
```

```

JOIN Staff s_doc ON s_doc.id = sh.doctor_id
JOIN Doctors d_sup ON d_sup.staff_id = sh.supervisor_id
JOIN Staff s_sup ON s_sup.id = sh.supervisor_id
ORDER BY doctor_id, level;

```

Χρησιμοποιούμε Recursive Common Table Expression ώστε να εμφανίσουμε την ιεραρχία εποπτείας μεταξύ των ιατρών. Αυτό λειτουργεί ως εξής:

Αρχικά κρατάμε τους ιατρούς που έχουν άμεσο επόπτη και τους τοποθετούμε στο επίπεδο 1 της εποπτείας. Στη συνέχεια, το αναδρομικό μέρος του query ανεβαίνει σταδιακά στην αλυσίδα εποπτείας, βρίσκοντας τον επόπτη του επόπτη και αυξάνοντας κάθε φορά το επίπεδο κατά 1. Έτσι εμφανίζονται όχι μόνο οι άμεσοι επόπτες, αλλά και τα ανώτερα επίπεδα εποπτείας. Τέλος, γίνεται σύνδεση με τους πίνακες Staff και Doctors, ώστε να εμφανιστούν τα ονόματα και οι βαθμίδες τόσο των ιατρών όσο και των εποπτών τους.

- **Query 14:**

Βρείτε ποιες κατηγορίες ICD-10 διαγνώσεων είχαν τον ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη, με τουλάχιστον 5 περιστατικά ανά έτος.

```

WITH counts_per_year AS
(
    SELECT
        LEFT(h.icd10_entry_code, 3) AS icd_category,
        YEAR(h.entry_date) AS year,
        COUNT(*) AS admission_count
    FROM Hospitalizations h
    WHERE h.icd10_entry_code IS NOT NULL
    GROUP BY LEFT(h.icd10_entry_code, 3), YEAR(h.entry_date)
    HAVING COUNT(*) >= 5
)
SELECT
    y1.icd_category AS 'Κατηγορία ICD-10',
    y1.year AS 'Έτος 1',
    y2.year AS 'Έτος 2',
    y1.admission_count AS 'Αριθμός εισαγωγών'
FROM counts_per_year y1
JOIN counts_per_year y2
    ON y1.icd_category = y2.icd_category
    AND y2.year = y1.year +1
    AND y1.admission_count = y2.admission_count
ORDER BY y1.icd_category, y1.year;

```

Στο Query 14 χρησιμοποιούμε ένα CTE με όνομα counts_per_year ώστε να υπολογίσουμε αρχικά τον αριθμό εισαγωγών ανά κατηγορία ICD-10 και ανά έτος. Η κατηγορία ICD-10 προκύπτει από τα τρία πρώτα ψηφία του κωδικού διάγνωσης εισόδου, μέσω της συνάρτησης LEFT(h.icd10_entry_code, 3).

Με το GROUP BY ομαδοποιούμε τις νοσηλείες ανά κατηγορία και έτος, ενώ με το HAVING COUNT(*) >= 5 κρατάμε μόνο τις κατηγορίες που έχουν τουλάχιστον 5 εισαγωγές. Στη συνέχεια, κάνουμε JOIN το CTE με τον εαυτό του, ώστε να βρούμε κατηγορίες ICD-10 που εμφανίζονται σε δύο διαδοχικά έτη με τον ίδιο ακριβώς αριθμό εισαγωγών. Τέλος, τα αποτελέσματα ταξινομούνται με βάση την κατηγορία ICD-10 και το πρώτο έτος.

❖ Σημείωση για τα Fake Data:

Το συγκεκριμένο query θέλουμε να εμφανίζει κατηγορίες ICD-10 που έχουν ίδιο αριθμό εισαγωγών σε δύο διαδοχικά έτη. Για να μπορεί να προκύψει τέτοιο αποτέλεσμα, στο script fake_data.py ενισχύονται συγκεκριμένες κατηγορίες ICD-10 για τα έτη 2025 και 2026, ώστε να εμφανίζονται αρκετές εισαγωγές ανά κατηγορία και έτος. Με αυτόν τον τρόπο το ερώτημα μπορεί να επιστρέψει κατηγορίες που πληρούν το όριο των τουλάχιστον 5 εισαγωγών και έχουν ίδιο πλήθος σε δύο συνεχόμενα έτη.

• **Query 15:**

Βρείτε την κατανομή των περιστατικών triage ανά επίπεδο επείγοντος, με μέσο χρόνο αναμονής ανά επίπεδο, ποσοστό περιστατικών που οδήγησαν σε νοσηλεία και κατανομή παραπομπών ανά τμήμα.

```
WITH urgency_statistics AS
(
    SELECT
        urgency_level,
        COUNT(*) AS total_cases,
        ROUND(AVG(TIMESTAMPDIFF(MINUTE, arrival_time, service_time)), 1)
    AS avg_wait_minutes,
        ROUND(100.0 * SUM(CASE WHEN referral_status = 'Hospitalization'
    THEN 1 ELSE 0 END)/COUNT(*), 2) AS hosp_percentage
    FROM Triage_Entries
    GROUP BY urgency_level
)
SELECT
    us.urgency_level AS 'Επίπεδο Επείγοντος',
    us.total_cases AS 'Σύνολο Περιστατικών',
    us.avg_wait_minutes AS 'Μέσος Χρόνος Αναμονής (σε λεπτά)',
    us.hosp_percentage AS 'Ποσοστό Νοσηλείας',
    CASE WHEN d.name IS NULL THEN 'Χωρίς παραπομπή' ELSE d.name END AS
    'Τμήμα Παραπομπής',
    COUNT(h.id) AS 'Παραπομπές σε τμήμα'
    FROM Triage_Entries te
    JOIN urgency_statistics us ON us.urgency_level = te.urgency_level
    LEFT JOIN Hospitalizations h ON h.triage_id = te.id
    LEFT JOIN Departments d ON d.id = h.department_id
    GROUP BY us.urgency_level, us.total_cases, us.avg_wait_minutes,
    us.hosp_percentage, d.name
```

ORDER BY us.urgency_level, d.name;

Στο αυτό το query χρησιμοποιούμε ένα CTE με όνομα urgency_statistics, στο οποίο υπολογίζονται τα βασικά στατιστικά ανά επίπεδο επείγοντος: το πλήθος περιστατικών, ο μέσος χρόνος αναμονής και το ποσοστό που κατέληξε σε νοσηλεία.

Στη συνέχεια, τα στοιχεία αυτά συνδέονται με τους πίνακες Triage_Entries, Hospitalizations και Departments, ώστε να εμφανιστεί και το τμήμα παραπομπής κάθε περιστατικού. Με LEFT JOIN διατηρούνται και τα περιστατικά που δεν οδήγησαν σε νοσηλεία. Τέλος, τα αποτελέσματα ομαδοποιούνται ανά επίπεδο επείγοντος και τμήμα παραπομπής.

11. Web εφαρμογή

Στο πλαίσιο της εργασίας υλοποιήθηκε και μια απλή web εφαρμογή για πιο εύκολη αλληλεπίδραση με τη βάση δεδομένων. Η εφαρμογή αναπτύχθηκε με Flask και συνδέεται με τη MariaDB μέσω της βιβλιοθήκης mysql-connector-python. Για την εμφάνιση των σελίδων χρησιμοποιούνται Jinja2 templates, HTML και CSS.

Η εφαρμογή περιλαμβάνει βασική πλοήγηση προς τις σελίδες “Αρχική”, “Ιατροί”, “Ασθενείς” και “Queries”. Στη σελίδα των ιατρών εμφανίζεται λίστα με τους ήδη καταχωρημένους ιατρούς και υπάρχει φόρμα για την προσθήκη νέου ιατρού. Αντίστοιχα, στη σελίδα των ασθενών εμφανίζονται βασικά στοιχεία των ασθενών και υπάρχει φόρμα για την εγγραφή νέου ασθενή.

Στη σελίδα “Queries” ο χρήστης μπορεί να επιλέξει ένα από τα SQL ερωτήματα της εργασίας και να το εκτελέσει μέσα από το γραφικό περιβάλλον. Για ορισμένα queries υπάρχουν και πεδία παραμέτρων, όπως επιλογή ειδικότητας, τμήματος, ημερομηνίας ή ID ασθενή/ιατρού. Τα αποτελέσματα εμφανίζονται σε μορφή πίνακα, ώστε να είναι πιο εύκολη η ανάγνωση και ο έλεγχός τους.

Για να τρέξει η εφαρμογή, πρώτα δημιουργούμε τη βάση με το install.sql και στη συνέχεια φορτώνουμε τα δεδομένα με το load.sql. Έπειτα εγκαθιστούμε τις βιβλιοθήκες που χρειάζονται από το requirements.txt και εκτελούμε την εφαρμογή με την εντολή app.py. Η εφαρμογή ανοίγει τοπικά από τον browser στη διεύθυνση <http://127.0.0.1:5000>.

Αναφορά χρήσης LLM (Large Language Models)

Στην εργασία χρησιμοποιήθηκε LLM βοηθητικά στη συγγραφή της αναφοράς και συγκεκριμένα βοήθησε στη διατύπωση και οργάνωση ορισμένων κειμένων, ώστε οι περιγραφές των SQL queries, των triggers και των σχεδιαστικών επιλογών να είναι πιο καθαρές και κατανοητές.

Επιπλέον, χρησιμοποιήθηκε υποστηρικτικά σε σημεία debugging και troubleshooting κατά

την ενσωμάτωση και τον έλεγχο των triggers στη βάση.

Χρησιμοποιήθηκε επίσης για διευκρινίσεις και καθοδήγηση σε βοηθητικά Python scripts που σχετίζονται με τη δημιουργία και φόρτωση δεδομένων.

Η τελική δομή της βάσης, οι πίνακες, τα queries, τα sql scripts, τα δεδομένα και ο έλεγχος των αποτελεσμάτων έγιναν από την ομάδα.

Παράρτημα: DDL script

Στο παρόν παράρτημα παρατίθεται το DDL script που χρησιμοποιήθηκε για την υλοποίηση της βάσης μας. Το script περιλαμβάνει τους ορισμούς των πινάκων, τα primary και foreign keys, τα CHECK και UNIQUE constraints, καθώς και τα indexes.

```
-- Database creation
-- CREATE DATABASE `hospital_db` /*!40100 DEFAULT CHARACTER SET
utf8mb4 COLLATE utf8mb4_general_ci */;
DROP DATABASE IF EXISTS hospitaldb;
CREATE DATABASE hospitaldb;
use hospitaldb;

-- Tables Creation
CREATE TABLE Staff (
    id INT AUTO_INCREMENT PRIMARY KEY,
    amka CHAR(11) NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    birth_date DATE NOT NULL,
    email VARCHAR(255) NOT NULL,
    phone VARCHAR(15) NOT NULL,
    hire_date DATE NOT NULL,
    staff_type VARCHAR(15) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    active BOOLEAN DEFAULT TRUE,
    CONSTRAINT chk_staff_type CHECK (staff_type IN ('doctor',
'nurse', 'admin'))
);

CREATE TABLE Patients (
    id INT AUTO_INCREMENT PRIMARY KEY,
    amka CHAR(11) NOT NULL,
```

```

first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
father_name VARCHAR(50) NOT NULL,
birth_date DATE NOT NULL,
gender VARCHAR(10) NOT NULL,
weight DECIMAL(5,2) NOT NULL,
height DECIMAL(3,2) NOT NULL,
address TEXT NOT NULL,
phone VARCHAR(15) NOT NULL,
email VARCHAR(255),
profession VARCHAR(255),
nationality VARCHAR(50) NOT NULL,
insurance_provider VARCHAR(100) NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,

CONSTRAINT chk_gender CHECK (gender IN ('Male', 'Female',
'Other'))
);

CREATE TABLE Doctors (
    staff_id INT PRIMARY KEY,
    license_number VARCHAR(20) NOT NULL,
    specialty VARCHAR(50),
    rank VARCHAR(30) NOT NULL,
    supervisor_id INT,

    FOREIGN KEY (staff_id) REFERENCES Staff(id) ON DELETE CASCADE,
    FOREIGN KEY (supervisor_id) REFERENCES Doctors(staff_id) ON
DELETE SET NULL,

CONSTRAINT chk_doctor_rank CHECK (rank IN ('Ειδικευόμενος',
'Επιμελητής Β', 'Επιμελητής Α', 'Διευθυντής')),

CONSTRAINT chk_supervisor CHECK (
    (rank = 'Ειδικευόμενος' AND supervisor_id IS NOT NULL) OR
    (rank = 'Διευθυντής' AND supervisor_id IS NULL) OR
    (rank NOT IN ('Ειδικευόμενος', 'Διευθυντής'))
)
);

```

```

CREATE TABLE Departments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    bed_count INT NOT NULL DEFAULT 0,
    location VARCHAR(100),
    head_doctor_id INT,
    min_doctors INT DEFAULT 3,
    min_nurses INT DEFAULT 6,
    min_admins INT DEFAULT 2,

    FOREIGN KEY (head_doctor_id) REFERENCES Doctors(staff_id) ON
DELETE SET NULL
);

CREATE TABLE Nurses (
    staff_id INT PRIMARY KEY,
    rank VARCHAR(50) NOT NULL,
    department_id INT NOT NULL,
    FOREIGN KEY (staff_id) REFERENCES Staff(id) ON DELETE CASCADE,
    FOREIGN KEY (department_id) REFERENCES Departments(id) ON
DELETE RESTRICT,

    CONSTRAINT chk_nurse_rank CHECK (rank IN ('Βοηθός Νοσηλεύτη',
'Νοσηλεύτης', 'Προϊστάμενος'))
);

CREATE TABLE Administrative_Staff (
    staff_id INT PRIMARY KEY,
    duty_role VARCHAR(100) NOT NULL,
    office_location VARCHAR(50),
    department_id INT NOT NULL,
    FOREIGN KEY (staff_id) REFERENCES Staff(id) ON DELETE CASCADE,
    FOREIGN KEY (department_id) REFERENCES Departments(id) ON
DELETE RESTRICT
);

CREATE TABLE Doctor_Departments (
    doctor_id INT NOT NULL,
    department_id INT NOT NULL,

    PRIMARY KEY (doctor_id, department_id),

```



```

    FOREIGN KEY (doctor_id) REFERENCES Doctors(staff_id) ON DELETE
CASCADE,
    FOREIGN KEY (department_id) REFERENCES Departments(id) ON
DELETE CASCADE
);

```

```

CREATE TABLE Beds (
    id INT AUTO_INCREMENT PRIMARY KEY,
    bed_number VARCHAR(20) NOT NULL,
    bed_type VARCHAR(50) NOT NULL,
    status VARCHAR(50) NOT NULL DEFAULT 'Διαθέσιμη',
    department_id INT NOT NULL,

    FOREIGN KEY (department_id) REFERENCES Departments(id) ON
DELETE RESTRICT,

    CONSTRAINT chk_bed_status CHECK (status IN ('Διαθέσιμη',
'Κατειλημμένη', 'Υπό συντήρηση'))
);

```

```

CREATE TABLE Emergency_Contacts (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    phone VARCHAR(15) NOT NULL,
    relationship VARCHAR(50),
    FOREIGN KEY (patient_id) REFERENCES Patients(id) ON DELETE
CASCADE
);

```

```

CREATE TABLE KEN_Ref (
    id INT AUTO_INCREMENT PRIMARY KEY,
    code VARCHAR(100) NOT NULL,
    base_cost DECIMAL(10,2) NOT NULL,
    mdn_days INT NOT NULL,
    INDEX (code)
);

```

```

CREATE TABLE ICD10_Ref (
    id INT AUTO_INCREMENT PRIMARY KEY,

```

```

        code VARCHAR(100) NOT NULL,
        description TEXT NOT NULL,
        INDEX (code)
    );

CREATE TABLE Triage_Entries (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    nurse_id INT NOT NULL,
    service_time DATETIME,
    arrival_time DATETIME NOT NULL,
    symptoms TEXT NOT NULL,
    urgency_level INT NOT NULL CHECK (urgency_level BETWEEN 1 AND
5),
    referral_status VARCHAR(50) NOT NULL,
    FOREIGN KEY (patient_id) REFERENCES Patients(id) ON DELETE
RESTRICT,
    FOREIGN KEY (nurse_id) REFERENCES Nurses(staff_id) ON DELETE
RESTRICT,
    CONSTRAINT chk_referral CHECK (referral_status IN ('Exit',
'Hospitalization'))
);

CREATE TABLE Hospitalizations (
    id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    bed_id INT NOT NULL,
    department_id INT NOT NULL,
    entry_date DATETIME NOT NULL,
    exit_date DATETIME,
    icd10_entry_code VARCHAR(100),
    icd10_exit_code VARCHAR(100),
    ken_code VARCHAR(100),
    total_cost DECIMAL(10,2),
    triage_id INT NOT NULL UNIQUE,

    FOREIGN KEY (patient_id) REFERENCES Patients(id) ON DELETE
RESTRICT,
    FOREIGN KEY (bed_id) REFERENCES Beds(id) ON DELETE RESTRICT,
    FOREIGN KEY (department_id) REFERENCES Departments(id) ON
DELETE RESTRICT,
    FOREIGN KEY (icd10_entry_code) REFERENCES ICD10_Ref(code) ON

```

```

DELETE RESTRICT,
    FOREIGN KEY (icd10_exit_code) REFERENCES ICD10_Ref(code) ON
DELETE RESTRICT,
    FOREIGN KEY (ken_code) REFERENCES KEN_Ref(code) ON DELETE
RESTRICT,
    FOREIGN KEY (triage_id) REFERENCES Triage_Entries(id) ON
DELETE RESTRICT
);

CREATE TABLE Active_Substances (
    id INT AUTO_INCREMENT PRIMARY KEY,
    substance_name TEXT
);

CREATE TABLE Medications (
    id INT AUTO_INCREMENT PRIMARY KEY,
    product_name TEXT
);

CREATE TABLE Medication_Substances (
    medication_id INT NOT NULL,
    substance_id INT NOT NULL,
    PRIMARY KEY (medication_id, substance_id),
    FOREIGN KEY (medication_id) REFERENCES Medications(id) ON
DELETE CASCADE,
    FOREIGN KEY (substance_id) REFERENCES Active_Substances(id) ON
DELETE RESTRICT
);

CREATE TABLE Patient_Allergies (
    patient_id INT NOT NULL,
    substance_id INT NOT NULL,
    PRIMARY KEY (patient_id, substance_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(id) ON DELETE
CASCADE,
    FOREIGN KEY (substance_id) REFERENCES Active_Substances(id) ON
DELETE RESTRICT
);

CREATE TABLE Prescriptions (
    id INT AUTO_INCREMENT PRIMARY KEY,
    doctor_id INT NOT NULL,

```

```

    patient_id INT NOT NULL,
    medication_id INT NOT NULL,
    hospitalization_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    dosage VARCHAR(50),
    frequency VARCHAR(50),
    FOREIGN KEY (doctor_id) REFERENCES Doctors(staff_id) ON DELETE
RESTRICrT,
    FOREIGN KEY (patient_id) REFERENCES Patients(id) ON DELETE
RESTRICrT,
    FOREIGN KEY (medication_id) REFERENCES Medications(id) ON
DELETE RESTRICrT,
    FOREIGN KEY (hospitalization_id) REFERENCES
Hospitalizations(id) ON DELETE RESTRICrT,
    UNIQUE KEY unique_prescription (doctor_id, patient_id,
medication_id, start_date)
);

```

```

CREATE TABLE Shifts (
    id INT AUTO_INCREMENT PRIMARY KEY,
    department_id INT NOT NULL,
    shift_type VARCHAR(9) NOT NULL,
    shift_date DATE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    shift_status VARCHAR(10) NOT NULL DEFAULT 'scheduled',

    UNIQUE KEY department_shift_date (department_id, shift_date,
shift_type),
    FOREIGN KEY (department_id) REFERENCES Departments(id) ON
DELETE RESTRICrT,
    CONSTRAINT chk_shifttype CHECK (shift_type IN ('Morning',
'Afternoon', 'Night')),
    CONSTRAINT chk_shiftstatus CHECK (shift_status IN
('scheduled', 'ongoing', 'completed', 'cancelled'))
);

```

```

CREATE TABLE Staff_Shifts (
    staff_id INT NOT NULL,
    shift_id INT NOT NULL,

```

```

start_time TIME,
end_time TIME,
started_date DATE,
PRIMARY KEY (staff_id, shift_id),
FOREIGN KEY (staff_id) REFERENCES Staff(id) ON DELETE CASCADE,
FOREIGN KEY (shift_id) REFERENCES Shifts(id) ON DELETE CASCADE
);

```

```

CREATE TABLE Shift_Monthly_Limits (
    id INT PRIMARY KEY AUTO_INCREMENT,
    staff_id INT NOT NULL,
    ml_year INT NOT NULL,
    ml_month INT NOT NULL CHECK (ml_month BETWEEN 1 AND 12),
    ml_num INT DEFAULT 0,
    FOREIGN KEY (staff_id) REFERENCES Staff(id) ON DELETE CASCADE,
    UNIQUE KEY staff_ml_month (staff_id, ml_year, ml_month)
);

```

```

CREATE TABLE LabExam_Ref (
    id INT AUTO_INCREMENT PRIMARY KEY,
    code VARCHAR(100) NOT NULL,
    name TEXT,
    category VARCHAR(255) NOT NULL,
    INDEX (code),
    CONSTRAINT chk_lab_ref_category
    CHECK (category IN (
        'Γ. ΑΠΕΙΚΟΝΙΣΗ - ΕΠΕΜΒΑΤΙΚΕΣ ΚΑΙ ΘΕΡΑΠΕΥΤΙΚΕΣ
ΑΚΤΙΝΙΚΕΣ ΠΡΑΞΕΙΣ',
        'Δ. ΠΡΑΞΕΙΣ ΒΙΟΠΑΘΟΛΟΓΙΑΣ',
        'Ε. ΠΡΑΞΕΙΣ ΙΑΤΡΟΔΙΚΑΣΤΙΚΗΣ - ΠΑΘΟΛΟΓΙΚΗΣ ΑΝΑΤΟΜΙΚΗΣ
- ΚΥΤΤΑΡΟΛΟΓΙΑΣ'
    ))
);

```

```

CREATE TABLE LabExam (
    id INT AUTO_INCREMENT PRIMARY KEY,
    code VARCHAR(100) NOT NULL,
    exam_date DATETIME NOT NULL,
    result TEXT,
    result_value DECIMAL(10,3),
    result_unit VARCHAR(20),

```

```

        cost DECIMAL(10,2) NOT NULL,
        doctor_id INT,
        hospitalization_id INT NOT NULL,
        FOREIGN KEY (code) REFERENCES LabExam_Ref(code) ON DELETE
RESTRICT,
        FOREIGN KEY (doctor_id) REFERENCES Doctors(staff_id) ON DELETE
SET NULL,
        FOREIGN KEY (hospitalization_id) REFERENCES
Hospitalizations(id) ON DELETE RESTRICT
);

```

```

CREATE TABLE Operating_Rooms (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    room_type TEXT NOT NULL
);

```

-- Προσθέτω πίνακα για τους κωδικούς των επεμβάσεων

```

CREATE TABLE MedicalAct_Ref (
    id INT AUTO_INCREMENT PRIMARY KEY,
    code VARCHAR(100) NOT NULL,
    name TEXT,
    category VARCHAR(255) NOT NULL,
    INDEX (code),
    CONSTRAINT chk_act_ref_category
CHECK (category IN (
        'Α. ΠΡΑΞΕΙΣ ΑΙΝΑΙΣΘΗΣΙΑΣ',
        'Β. ΠΡΑΞΕΙΣ ΧΕΙΡΟΥΡΓΙΚΕΣ - ΕΠΕΜΒΑΤΙΚΕΣ -
ΕΝΔΟΣΚΟΠΙΚΕΣ'
    ))
);

```

```

CREATE TABLE Medical_Acts (
    id INT AUTO_INCREMENT PRIMARY KEY,
    act_code VARCHAR(100) NOT NULL,
    duration_minutes INT NOT NULL,
    cost DECIMAL(10,2) NOT NULL,
    scheduled_time DATETIME NOT NULL,
    hospitalization_id INT NOT NULL,
    room_id INT NOT NULL,
    main_doctor_id INT NOT NULL,
    FOREIGN KEY (hospitalization_id) REFERENCES

```

```
Hospitalizations(id) ON DELETE RESTRICT,
    FOREIGN KEY (room_id) REFERENCES Operating_Rooms(id) ON DELETE
    RESTRICT,
    FOREIGN KEY (main_doctor_id) REFERENCES Doctors(staff_id) ON
    DELETE RESTRICT,
    FOREIGN KEY (act_code) REFERENCES MedicalAct_Ref(code) ON
    DELETE RESTRICT
);
```

```
CREATE TABLE Medical_Act_Assistants (
    act_id INT NOT NULL,
    staff_id INT NOT NULL,
    PRIMARY KEY (act_id, staff_id),
    FOREIGN KEY (act_id) REFERENCES Medical_Acts(id) ON DELETE
    CASCADE,
    FOREIGN KEY (staff_id) REFERENCES Staff(id) ON DELETE CASCADE
);
```

```
CREATE TABLE Hospitalization_Ratings (
    hospitalization_id INT PRIMARY KEY,
    nursing_care_quality TINYINT CHECK (nursing_care_quality
    BETWEEN 1 AND 5),
    cleanliness TINYINT CHECK (cleanliness BETWEEN 1 AND 5),
    food_quality TINYINT CHECK (food_quality BETWEEN 1 AND 5),
    overall_experience TINYINT CHECK (overall_experience BETWEEN 1
    AND 5),
    FOREIGN KEY (hospitalization_id) REFERENCES
    Hospitalizations(id) ON DELETE RESTRICT
);
```

```
CREATE TABLE Doctor_Ratings (
    hospitalization_id INT,
    doctor_id INT,
    medical_care_quality TINYINT CHECK (medical_care_quality
    BETWEEN 1 AND 5),
    PRIMARY KEY (hospitalization_id, doctor_id),
    FOREIGN KEY (hospitalization_id) REFERENCES
    Hospitalizations(id) ON DELETE RESTRICT,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(staff_id) ON DELETE
    RESTRICT
);
```

```

CREATE TABLE Images (
    id INT AUTO_INCREMENT PRIMARY KEY,
    image_url VARCHAR(255) NOT NULL,
    description TEXT,
    doctor_id INT,
    nurse_id INT,
    admin_id INT,
    department_id INT,
    room_id INT,

    FOREIGN KEY (doctor_id) REFERENCES Doctors(staff_id) ON DELETE
    CASCADE,
    FOREIGN KEY (nurse_id) REFERENCES Nurses(staff_id) ON DELETE
    CASCADE,
    FOREIGN KEY (admin_id) REFERENCES
    Administrative_Staff(staff_id) ON DELETE CASCADE,
    FOREIGN KEY (department_id) REFERENCES Departments(id) ON
    DELETE CASCADE,
    FOREIGN KEY (room_id) REFERENCES Operating_Rooms(id) ON DELETE
    CASCADE,

    CONSTRAINT chk_image_entity CHECK (
        (doctor_id IS NOT NULL) +
        (nurse_id IS NOT NULL) +
        (admin_id IS NOT NULL) +
        (department_id IS NOT NULL) +
        (room_id IS NOT NULL) = 1
    )
);

-- INDEXES
-- Staff
CREATE INDEX idx_staff_name ON Staff(last_name, first_name);
CREATE INDEX idx_staff_type ON Staff(staff_type);
-- Patients
CREATE INDEX idx_patients_name ON Patients(last_name, first_name);
CREATE INDEX idx_patients_insurance ON
Patients(insurance_provider);
-- Doctors
CREATE INDEX idx_doctors_specialty ON Doctors(specialty);
CREATE INDEX idx_doctors_rank ON Doctors(rank);
CREATE INDEX idx_doctors_supervisor ON Doctors(supervisor_id);

```



```

-- Departments
CREATE INDEX idx_departments_head ON Departments(head_doctor_id);
-- Beds
CREATE INDEX idx_beds_status ON Beds(status);
CREATE INDEX idx_beds_department ON Beds(department_id);
-- Triage_Entries
CREATE INDEX idx_triage_patient ON Triage_Entries(patient_id);
CREATE INDEX idx_triage_urgency ON Triage_Entries(urgency_level);
CREATE INDEX idx_triage_arrival ON Triage_Entries(arrival_time);
-- Hospitalizations
CREATE INDEX idx_hosp_patient ON Hospitalizations(patient_id);
CREATE INDEX idx_hosp_department ON
Hospitalizations(department_id);
CREATE INDEX idx_hosp_dates ON Hospitalizations(entry_date,
exit_date);
CREATE INDEX idx_hosp_ken ON Hospitalizations(ken_code);
CREATE INDEX idx_hosp_icd10 ON Hospitalizations(icd10_entry_code);
-- Shifts
CREATE INDEX idx_shifts_date ON Shifts(shift_date);
CREATE INDEX idx_shifts_type ON Shifts(shift_type);
CREATE INDEX idx_shifts_department ON Shifts(department_id,
shift_date);
-- Staff_Shifts
CREATE INDEX idx_staff_shifts_staff ON Staff_Shifts(staff_id);
CREATE INDEX idx_staff_shifts_shift ON Staff_Shifts(shift_id);
-- Shift_Monthly_Limits
CREATE INDEX idx_sml_staff_date ON Shift_Monthly_Limits(staff_id,
ml_year, ml_month);
-- Prescriptions
CREATE INDEX idx_presc_doctor ON Prescriptions(doctor_id);
CREATE INDEX idx_presc_patient ON Prescriptions(patient_id);
CREATE INDEX idx_presc_medication ON Prescriptions(medication_id);
CREATE INDEX idx_presc_hospitalization ON
Prescriptions(hospitalization_id);
CREATE INDEX idx_presc_dates ON Prescriptions(start_date,
end_date);
-- Medications & Substances
CREATE INDEX idx_medsubst_medication ON
Medication_Substances(medication_id);
CREATE INDEX idx_medsubst_substance ON
Medication_Substances(substance_id);
-- Patient_Allergies

```

```

CREATE INDEX idx_allergies_patient ON
Patient_Allergies(patient_id);
CREATE INDEX idx_allergies_substance ON
Patient_Allergies(substance_id);
-- Medical_Acts
CREATE INDEX idx_medacts_doctor ON Medical_Acts(main_doctor_id);
CREATE INDEX idx_medacts_room ON Medical_Acts(room_id);
CREATE INDEX idx_medacts_time ON Medical_Acts(scheduled_time);
-- LabExam
CREATE INDEX idx_labexam_date ON LabExam(exam_date);
CREATE INDEX idx_labexam_hospitalization ON
LabExam(hospitalization_id);
CREATE INDEX idx_labexam_doctor ON LabExam(doctor_id);
-- Doctor_Ratings
CREATE INDEX idx_docrat_doctor ON Doctor_Ratings(doctor_id);
CREATE INDEX idx_docrat_hosp ON
Doctor_Ratings(hospitalization_id);
-- Hospitalization_Ratings
CREATE INDEX idx_hosprat_hosp ON
Hospitalization_Ratings(hospitalization_id);

```